Multi Agents with LangGraph

Patterns, Communication and Deep Agents

General Course Remarks

- Course times
- Drinks
- Meals
- Toilettes
- Smoking
- Mobile Phones
- Manual
- Software

Everything should be made as simple as possible, but not simpler

Albert Einstein

Ja, was ich hier geschrieben habe, macht im Einzelnen überhaupt nicht den Anspruch auf Neuheit; und darum gebe ich auch keine Quellen an, weil es mir gleichgültig ist, ob das was ich gedacht habe, vor mir schon ein anderer gedacht hat.

Ludwig Wittgenstein

Lab Environment

- **Labs run as Jupyter notebooks on the classroom laptop :**
  - `uv sync` installs the pinned stack from `uv.lock`
  - `.env` holds the course key, outside every notebook
  - `uv run jupyter lab` from the repository root
- **Course model `gpt-6-luna` through OpenRouter ; no local model needed**
- **Google Colab runs the same notebooks as a fallback**
- **`check_setup.py` confirms the stack and the key in under a minute**

SpiralTrain developer courses

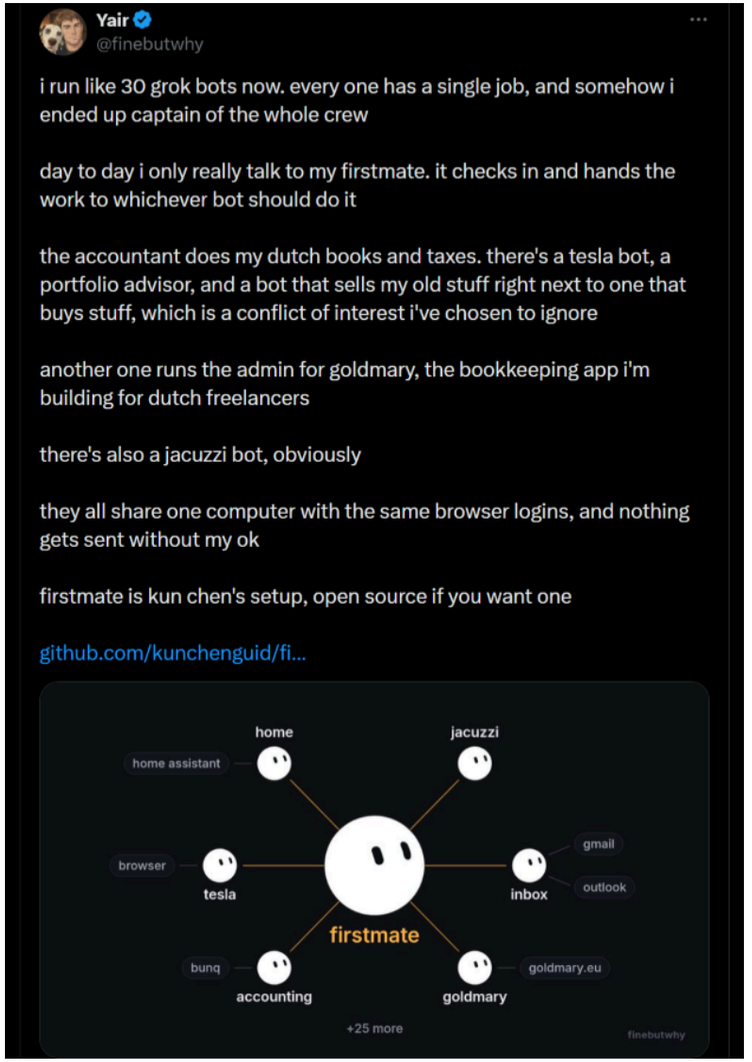
- ▶ **Introduction to LangGraph**
- Graph Fundamentals
- State Management
- Building Agents
- Multi-Agent Patterns
- Agent Communication
- Advanced Workflows
- Deep Agents
- Production Deployment
- Real-World Applications

- A Crew of Thirty Agents
- Ten Thousand Agents on Navier-Stokes
- How the Swarm Was Organised
- A Research Team of Agents
- Recursive Language Models
- The Bill for a Swarm
- Patterns Behind the Examples
- What is LangGraph?
- Runtime and Harness Layers
- Agent Frameworks Overview
- Framework Capabilities
- LangGraph Control Flow
- Model APIs and Agent Runtimes
- Why Multi-Agent?
- Context Engineering
- One Agent with More Tools
- When One Agent is Enough
- Graph-Based Architecture
- Conditional Routing
- A First Graph
- A First Multi-Agent System
- A StateGraph by Hand
- create_agent
- create_deep_agent
- Three Levels Compared
- Agents versus Workflows
- What Survived in 2026
- The firstmate Orchestrator
- The Service Desk Case
- The Capstone in Five Stages
- The Three Days

SpiralTrain developer courses

A Crew of Thirty Agents

- **A personal setup, October 2026, built on the open-source `firstmate` distribution :**
  - About 30 Grok agents, each with a single job
  - One first mate takes every request and hands it to the right agent
- **The jobs include :**
  - Dutch bookkeeping and taxes against a bunq account
  - An inbox over Gmail and Outlook, Home Assistant, a Tesla
  - The admin of his app goldmary.eu, a portfolio advisor
  - A seller and a buyer, side by side
- **The agents share one computer and one set of browser logins**
- **”nothing gets sent without my ok”**



SpiralTrain developer courses

Ten Thousand Agents on Navier-Stokes

- **OpenAI, 8 September 2026 : a proposed resolution of a Millennium Prize Problem :**
  - A smooth fluid at rest, under a smooth force, develops a singularity in finite time
- **Found by a system of coordinating agents on an unreleased internal model :**
  - On the order of 10,000 concurrent agents in the group that found it
  - 88 hours to the proof, then 17 hours to formalise it in Lean
  - 2.7 million messages and about 130 billion output tokens
- **Mathematicians are still examining it**
- **OpenAI does not claim the prize**

SpiralTrain developer courses

How the Swarm Was Organised

- Agents worked in groups and messaged only within their own group
- Separate groups received different variants of the problem :
  - Some aimed at a proof of smoothness, others at a blowup
- Codex merged the most useful findings of each group into new prompts
- Agents moved to Navier-Stokes after about 100 agents resolved a related Euler problem
- The model was replaced by a further-trained version during the run
- Lean checked the final proof :
  - The proof checker accepts no step that does not follow

SpiralTrain developer courses

A Research Team of Agents

- **Anthropic, June 2025 : a lead agent that spawns parallel subagents for research :**
  - Claude Opus 4 as the lead, Claude Sonnet 4 as the subagents
  - 90.2 % better than a single Opus 4 agent on Anthropic's internal research evaluation
- **The cost, from the same report :**
  - Agents use about 4 times the tokens of a chat ; multi-agent systems about 15 times
  - Token usage alone explained 80 % of the variance in performance
- **A poor fit, from the same report :**
  - Domains where every agent needs the same context
  - Most coding tasks, with fewer parallel parts than research

SpiralTrain developer courses

Recursive Language Models

- **A model that treats a long input as an environment to explore in code :**
  - The input sits in a Python REPL instead of the prompt
  - The model writes code that splits it and calls fresh copies of itself on the parts
  - Only the sub-calls use tools ; the root model sees their results
- **Zhang, Kraska and Khattab, MIT, December 2025 :**
  - Inputs about two orders of magnitude beyond the context window
- **Prime Intellect called it the paradigm of 2026**
- **Alex Zhang : a harness whose only tool is code**
- **Deep Agents ships the idea as dynamic subagents**

SpiralTrain developer courses

The Bill for a Swarm

- Every extra agent reads context and writes messages
- Measured in this course on a two-part question, `gpt-6-luna` :
  - A single agent answered in 2 model calls ; each multi-agent pattern needed 4 to 6
  - A Deep Agent read 28 times the input tokens of the single agent
- Alex Zhang suspects most of a large swarm's work is wasted
- Cognition, June 2025 : parallel agents with partial context make conflicting decisions
- The LangChain docs : a single agent with the right tools often does as well

Patterns Behind the Examples

example	pattern	module
a first mate and its crew	subagents behind one point of contact	05
groups that message only internally	scoped communication, hierarchical teams	05, 06
proof and disproof groups in parallel	parallel exploration, adversarial verification	05
Codex merging each group's findings	a synthesiser over map-reduce	05, 06
a seller and a buyer for one owner	conflict resolution	06
"nothing gets sent without my ok"	approval flows	07
Lean checking the proof	a checker written in code	05, 07
a recursive language model	dynamic subagents	08

SpiralTrain developer courses

What is LangGraph?

- **LangGraph is a runtime for stateful workflows and agents :**
  - Nodes do the work, such as a lookup, a model call or a whole agent
  - Edges fix which node may run next
- **The runtime adds services for long-running work :**
  - Checkpoints save state, so a stopped run resumes where it stopped
  - Interrupts pause a run until a person decides, such as approving a part order
  - Streaming shows progress and model output while the run continues
- **LangChain agents and Deep Agents run on LangGraph**
- **One graph can mix fixed steps with model decisions**

SpiralTrain developer courses

Runtime and Harness Layers

layer	provides	package
runtime	state, checkpoints, interrupts, streaming	LangGraph 1.2.15
framework	model interfaces, tools, agent loop, middleware	LangChain 1.4.5
harness	filesystem, delegation, context management	Deep Agents 0.7.24

- A framework supplies building blocks ; the application assembles them
- A harness supplies a ready agent, with tools and instructions around its loop
- The runtime leaves the most control to the application ; the harness the least

SpiralTrain developer courses

Agent Frameworks Overview

- **Provider-led SDKs integrate with their ecosystems :**
  - OpenAI Agents SDK, Google ADK and Claude Agent SDK
  - Support for other models varies by SDK and feature
- **Model-agnostic frameworks span providers :**
  - LangChain, CrewAI and Pydantic AI
  - LangGraph exposes the graph runtime directly
- **Microsoft Agent Framework succeeds AutoGen and Semantic Kernel**
- **Two requirements decide the choice :**
  - Who controls the flow, the code or the model
  - Where the agent runs and who operates it

SpiralTrain developer courses

Framework Capabilities

package	version	main abstraction
CrewAI	1.15.27	agents, crews, tasks and flows
OpenAI Agents SDK	0.23.1	agent loop, tools and handoffs
Google ADK	2.12.0	agents and workflow composition
Microsoft Agent Framework	1.21.0	agents and graph workflows
Claude Agent SDK	0.2.165	Claude Code harness as an SDK
Pydantic AI	2.55.0	typed dependencies and validated output

- LangGraph needs no agent loop ; a graph of plain Python functions runs as it is

SpiralTrain developer courses

LangGraph Control Flow

- **LangGraph starts from state and transitions :**
  - A node runs a function, a model call or a whole agent
  - The graph fixes which node may follow which
- **Agent SDKs start from an agent loop and its tools**
- **Other frameworks add explicit flows next to the loop :**
  - CrewAI provides Flows
  - ADK and Microsoft Agent Framework provide workflow composition
- **`create_agent` supplies a standard loop for an agent that needs no custom graph**

SpiralTrain developer courses

Model APIs and Agent Runtimes

interface	who runs the tool loop	role
Chat Completions	application	course OpenRouter model endpoint
Responses API	application, with hosted tools available	OpenAI model interface
Agents SDK	SDK in the application	agent loop, handoffs and tracing
Managed Deep Agents	provider-hosted harness	hosted sessions and execution

- **OpenAI-compatible names an API shape ; each provider supports its own subset**
- **A model only requests a tool call ; the runtime executes it and returns the result**
- **LangGraph keeps the application's state between model calls**

SpiralTrain developer courses

Why Multi-Agent?

- **A second agent gives a task its own context :**
  - The parts agent sees stock tools ; the manuals agent sees repair procedures
  - Each worker returns only its result to the orchestrator
- **Distributed development lets separate teams own separate agents**
- **Parallelism runs independent lookups, such as stock and manuals, at the same time**
- **LangChain documents three reasons to split :**
  - Too many tools cause poor tool selection
  - Specialised knowledge takes up extensive context
  - Sequential constraints unlock tools in stages, such as triage before repair advice

SpiralTrain developer courses

Context Engineering

- **Context engineering determines which information each agent sees :**
  - Subagents : each worker sees the task the supervisor wrote
  - Handoffs : the next specialist receives transferred state and history
  - Skills : one agent loads specialised prompts on demand
  - Router : each agent sees the part of the input routed to it
- **Veenhaven Bus specialists receive the identifiers and tools needed for their task**
- **Modules 05 and 06 compare calls and tokens across patterns**

SpiralTrain developer courses

One Agent with More Tools

- One agent with the right tools is the baseline every split is measured against
- Every tool description travels with every model call
- The Veenhaven Bus probe adds tools for unrelated company systems :
  - Eight repair tools and six questions stay the same
  - The added tools return no records
- Input tokens per call grow from 555 at 8 tools to 1,669 at 60
- The saved run misses one expected fact at 40 tools and none at 60
- No distractor tool is called in the saved run

demo01
one-agent-limits

When One Agent is Enough

- The LangChain docs warn before they teach the patterns :
  - ”a single agent with the right (sometimes dynamic) tools and prompt can often achieve similar results”
- One agent suits a task whose context fits and whose tools it picks correctly
- Dynamic tool loading keeps one agent’s tool list short
- A fixed process fits a workflow, with model calls only where judgment is needed
- A second agent adds model calls ; the course probe measured 2 for one agent, 4 to 6 per pattern
- A split needs a measured failure that one agent cannot fix
- Independent tasks with distinct context are the case worth testing

SpiralTrain developer courses

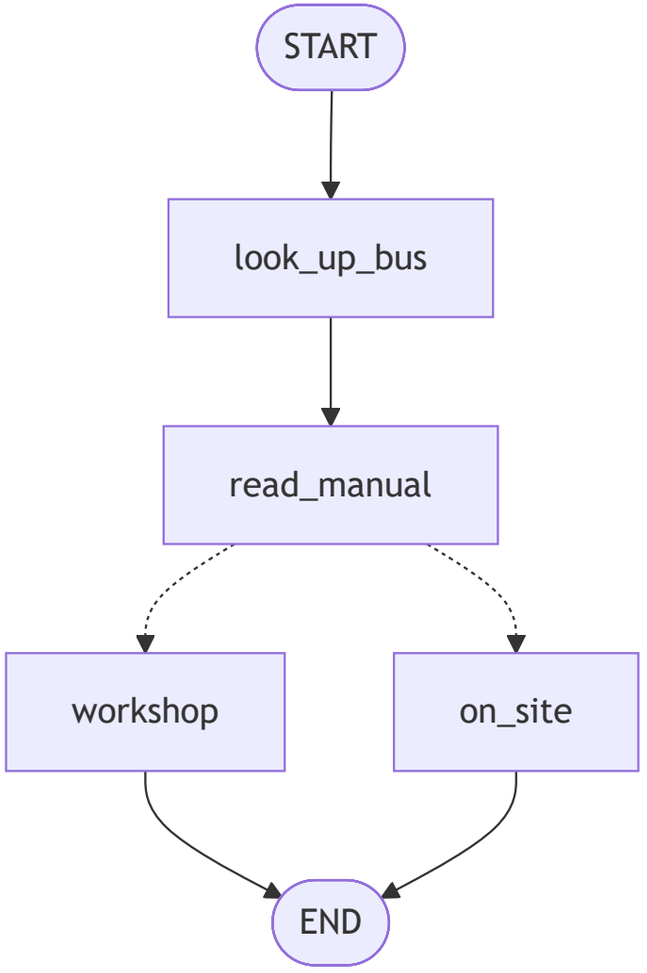
Graph-Based Architecture

- A state graph runs steps over one shared state, such as a bus report and its lookups
- Each node receives the state and returns the keys it changed
- Edges define execution order :
  - Fixed edges select a known next step
  - Conditional edges choose from state, such as workshop or on-site repair
- A cycle lets a step run again, such as a retry
- `compile` turns the definition into a graph that runs
- `START` and `END` mark the entry and the exit

Conditional Routing

- A routing function reads the state and returns the next node's name
- `Literal` in the return type names the possible targets
- The graph diagram shows a dashed edge to each possible target

```
def route(state: Report) -> Literal["workshop", "on_site"]:
    return "workshop" if "workshop" in state["action"] else "on_site"
```



SpiralTrain developer courses

A First Graph

```
builder = StateGraph(Report)
builder.add_node("look_up_bus", look_up_bus)
builder.add_node("read_manual", read_manual)
builder.add_node("workshop", lambda state: {"where": "workshop"})
builder.add_node("on_site", lambda state: {"where": "on site"})
builder.add_edge(START, "look_up_bus")
builder.add_edge("look_up_bus", "read_manual")
builder.add_conditional_edges("read_manual", route)
builder.add_edge("workshop", END)
builder.add_edge("on_site", END)
graph = builder.compile()
```

- The graph uses deterministic bus lookups and state updates, with no model call
- A node returns only the keys it changes, and LangGraph merges them into the state

demo02
first-look

SpiralTrain developer courses

A First Multi-Agent System

- A supervisor exposes specialists as tools :
  - A manuals agent reads procedures
  - A parts agent checks stock
- Each specialist receives the supervisor's task in a fresh conversation
- The supervisor writes the final answer from the specialists' tool results
- Both specialists use the same model ; neither sees the other's conversation
- The first saved run read fault code F-CE-110 as a part number
- The fix gives the parts agent `bus_info` and rules for fleet numbers and part numbers

demo02b
first-multi-agent

SpiralTrain developer courses

A StateGraph by Hand

- **The application defines the path for one bus diagnosis :**
  - A model extracts the fleet number and error code
  - Python looks up the bus, manual and spare part
  - A model writes the answer from those facts
- **State stores the extracted fields, facts and answer**
- **Code decides which lookups run and in what order**
- **The saved run makes 2 model calls, one to extract and one to answer**

demo03
three-levels

SpiralTrain developer courses

create_agent

- `create_agent` builds a tool-calling loop on LangGraph
- The model receives the prompt and tool schemas
- Each tool result goes back to the model for its next step
- The model decides which lookups to run and when to answer
- The bus question and the service desk tools are the same as in the graph by hand
- The saved run takes 4 model calls against 2 for the graph by hand
- The compiled graph exposes its nodes and edges

demo03b
create-agent

SpiralTrain developer courses

create_deep_agent

- `create_deep_agent` adds a harness around the agent loop :
  - Filesystem tools support context offloading
  - A general-purpose subagent supports delegation
  - Summarisation manages conversation history
- The service desk prompt and tools stay the same
- Built-in tool descriptions raise input tokens from 1,962 to 9,732 on the bus question
- Todo planning is opt-in in version 0.7.24

demo03c
deep-agent

Three Levels Compared

level	who picks the steps	calls	input tokens	expected facts
`StateGraph` by hand	the code	2	216	pass
`create_agent`	the model	4	1,962	pass
`create_deep_agent`	the model, inside a harness	4	9,732	pass

- One run each on `gpt-6-luna`, 14 October 2026, on the F-MM-210 question
- All three identify MM-BT-02 as the auxiliary battery with zero stock
- Code fixes the graph's lookup sequence
- Both agents chose the same three tools ; the harness tools stayed unused
- Module 08 compares the harness on a longer task

SpiralTrain developer courses

Agents versus Workflows

- **A workflow follows paths defined in code :**
  - Branches and loops have explicit conditions
  - Model calls can handle selected steps
- **An agent chooses its next action from available tools :**
  - The model decides the lookup sequence
  - The model decides when it has enough information
- **Both receive the same four bus reports**
- **A fixed bus workflow escalates a report without an error code**
- **An agent can search the manuals by symptom instead**
- **Production systems often combine fixed and agentic steps**

demo04
workflow-or-agent

SpiralTrain developer courses

What Survived in 2026

- **The design that held up in practice is an orchestrator with isolated workers :**
  - One agent owns the task and combines results
  - Each worker receives a bounded brief and its own context
- **`firstmate` implements this shape around coding harnesses :**
  - Each task has its own git worktree and session
  - A watcher wakes the orchestrator on events
  - Briefs and state persist in files
- **Each worker's result still goes through review before it is merged**

The firstmate Orchestrator

- `firstmate` supplies instructions, skills, scripts and persistent state for coding harnesses
- The MIT-licensed project supports Claude Code, Codex and Pi

`firstmate`	in this course
a first mate dispatching to crewmates	subagents behind one orchestrator, module 05
one git worktree and session per task	context isolation and backends, module 08
a watcher that wakes the first mate on events	event systems and async subagents, modules 06 and 09
briefs and state files on disk	checkpointers and the `Store`, module 03
merges escalated to the human	interrupts and approval flows, module 07

- Yair reports using `firstmate` for his coding fleet

The Service Desk Case

- **Aurelia Imaging Systems is a fictional imaging-equipment manufacturer**
- **The capstone builds a service desk over its supplied data :**
  - Manuals and service bulletins
  - Parts, equipment and service records in SQLite
  - Tickets and an evaluation set
- **Specialists handle manuals, parts and scheduling**
- **Part orders pause for approval**
- **The demos run on a second fictional company, Veenhaven Bus :**
  - 10 buses at 5 depots, 13 parts, 31 manuals and 43 tickets

The Capstone in Five Stages

stage	after	Aurelia service desk
1	module 03	triage, a `service.db` lookup and an answer ; typed state and a checkpointer
2	module 05	specialists for parts, manuals and scheduling behind a supervisor
3	module 07	parallel diagnosis with a judge ; the manuals agent over A2A ; part-order approval inside a subagent
4	module 08	the same desk as a Deep Agent, compared with stage 3
5	module 09	serving with `langgraph dev`, an async diagnosis subagent and per-agent evaluation

- **Evaluation compares answer quality, model calls and tokens as the desk gains capabilities**

The Three Days

day	modules	builds
Monday	01 to 04	graphs, state and single agents from LangGraph primitives
Tuesday	05 to 07	the multi-agent patterns, communication between agents, control
Wednesday	08 to 10	the same system as a Deep Agent, then served, evaluated and applied

- **Three ideas run through every module :**
  - Multi-agent is context engineering
  - One agent first ; add an agent when a measurement says so
  - Build each pattern by hand, then use the harness

SpiralTrain developer courses

Summary : Introduction to LangGraph

- LangGraph runs stateful workflows and agents
- LangChain supplies agent abstractions ; Deep Agents supplies a harness
- State, nodes and edges make control flow explicit
- Workflows fix paths ; agents choose actions through tools
- Multi-agent design decides what each agent sees
- One agent sets the baseline for quality and cost
- Isolated workers suit tasks with distinct context needs
- A measurement decides whether a second agent pays for itself

SpiralTrain developer courses

- Introduction to LangGraph
- **Graph Fundamentals**
- State Management
- Building Agents
- Multi-Agent Patterns
- Agent Communication
- Advanced Workflows
- Deep Agents
- Production Deployment
- Real-World Applications

- Why LangGraph?
- LangChain vs. LangGraph
- Thinking in LangGraph
- Directed Graphs
- DAGs and Cycles
- Fan-out and Fan-in
- State Machines
- Model Transitions
- LangGraph Concepts
- Node Functions
- StateGraph Nodes and Edges
- Hello LangGraph
- START and END
- Conditional Edges
- Routing with path_map
- Loops and Cycles
- GraphRecursionError
- Command
- Parallel Branches and Send
- Deferred Nodes
- Compiling a Graph
- Visualising a Graph
- Functional API
- Graph API or Functional API
- A Model in a Node

SpiralTrain developer courses

Why LangGraph?

- **Chains suit sequential flow :** `prompt -> model -> parser -> result`
- **A work order needs more control than a chain :**
  - An urgent order goes to an engineer, a routine one to the queue
  - A failed diagnosis runs again
  - Depot reports run in parallel ; a part order waits for approval
- **LangGraph models a workflow as nodes over shared state :**
  - Edges express branching and loops
  - State is explicit and typed
- **LangChain agents and Deep Agents use the LangGraph runtime**
- **The application code owns the shape of the workflow**

LangChain vs. LangGraph

Concern	LCEL runnables	LangGraph
Composition	Sequences, parallel calls and branches	Nodes and edges over state
State	Values passed between runnables	Explicit state returned from nodes
Repetition	Application code or custom runnables	Cycles and conditional edges
State updates	Application-managed values	Explicit updates returned from nodes

- `create_agent` returns a compiled LangGraph graph
- A graph of its own suits logic specific to the application, such as a work order's lifecycle

SpiralTrain developer courses

Thinking in LangGraph

- **A design starts from the process :**
  - A work order is diagnosed, waits for parts, then is repaired
  - Diagnosis needs judgment ; the parts lookup does not
- **Each step becomes a node ; a model call for judgment, a plain function for fixed work**
- **State holds every value a step reads, writes or needs after a pause**
- **Edges connect the nodes :**
  - Fixed edges for known order, such as parts before repair
  - Conditional edges for decisions, such as a second diagnosis
- **Control points go where a mistake costs most, such as a part order**
- **The graph drawing documents the design**

SpiralTrain developer courses

Directed Graphs

- A directed graph contains vertices and directed edges :
  - Vertices are steps, such as `diagnose`, `parts` and `repair`
  - Edges are allowed transitions, such as `diagnose` to `parts`
- A path follows the direction of each edge
- Fan-out sends execution to several branches, such as one per depot
- Fan-in joins the branch results at a later step
- A directed acyclic graph has no path back to an earlier vertex
- A cyclic graph contains at least one loop

SpiralTrain developer courses

DAGs and Cycles

- A DAG contains no directed cycle
- A cyclic graph allows a path to return to an earlier node
- A work order that goes from open to diagnosed to waiting for parts to closed is a DAG
- An edge from `parts` back to `diagnose` for a retry adds a cycle
- LangGraph accepts both ; a recursion limit bounds every cycle

SpiralTrain developer courses

Fan-out and Fan-in

- Fan-out starts several branches from one step, such as a report per depot
- Fan-in continues once every branch has returned
- Branches that write one key need a reducer, such as `operator.add` on a list of reports
- `Send` creates branches whose number is known only at runtime, such as one per depot in the input

SpiralTrain developer courses

State Machines

- A state machine represents a process as states and transitions :
  - State records where the work order is, such as open or waiting for parts
  - A transition changes state and selects the next step
- A plain node applies a fixed transition, such as open to diagnosed
- A model can choose a transition when the next step needs judgment
- A cycle lets the work order return to diagnosis
- An attempts counter in state decides when the loop ends

demo01b
work-order-lifecycle

SpiralTrain developer courses

Model Transitions

- A deterministic node applies a fixed transition to state
- A model node supplies the judgment, such as whether a work order is electrical or mechanical
- A routing function maps that label to a node, such as the electrical bay
- The other steps stay plain functions

SpiralTrain developer courses

LangGraph Concepts

- A graph is a workflow over state
- State is a schema that nodes read and update
- A node is a function that receives state and returns updates
- An edge connects a node to a possible next node
- A conditional edge selects a target from the current state
- `START` and `END` mark graph entry and exit
- A builder is compiled before it is invoked

SpiralTrain developer courses

Node Functions

- A node receives the current state as its input
- A node returns only the keys it changed, such as `{"status": "diagnosed"}`
- A node may be a Python function, a runnable, or a compiled graph
- A node can return `Command` to update state and route execution

SpiralTrain developer courses

StateGraph Nodes and Edges

- `StateGraph` builds a graph around a state schema :
  - `add_node` registers a function or compiled graph
  - `add_edge` adds a fixed transition
  - `add_conditional_edges` adds a routed transition
- A node returns a partial state update
- A compiled graph exposes `invoke` and `stream`

demo02
nodes-and-edges

Hello LangGraph

```python
from typing import TypedDict
from langgraph.graph import END, START, StateGraph

class Number(TypedDict):
    value: int

builder = StateGraph(Number)
builder.add_node("double", lambda s: {"value": s["value"] * 2})
builder.add_edge(START, "double")
builder.add_edge("double", END)
graph = builder.compile()
print(graph.invoke({"value": 5}))
```

SpiralTrain developer courses

START and END

- `START` identifies the graph entry point
- `END` marks a completed path
- `add_edge(START, "node")` defines the entry edge
- `add_edge("node", END)` defines a terminal edge
- `set_entry_point` and `set_finish_point` provide named alternatives

demo02b
start-end-entry

SpiralTrain developer courses

Conditional Edges

- **A conditional edge calls a routing function at runtime :**
  - The function reads state and returns a route label
  - `route` returns `"page"` for an urgent work order and `"queue"` otherwise
- **Route labels can match node names directly**
- **A `Literal` return type documents the possible routes**
- **A route can select a branch or send execution around a cycle**

demo03
routing

SpiralTrain developer courses

Routing with path_map

- `path_map` maps route labels to graph node names :

```
builder.add_conditional_edges(
    "triage",
    route,
    {"urgent": "page_engineer", "routine": "queue"},
)
```

- A dictionary maps labels to destinations
- A list declares possible destinations when labels match node names

demo03b
path-map

SpiralTrain developer courses

Loops and Cycles

- A cycle returns execution to an earlier node :
  - A router decides whether to continue or exit
- Loops support retries and iterative refinement
- Every invocation has a recursion limit measured in graph steps
- Exceeding the limit raises `GraphRecursionError`
- Set `recursion_limit` in the invocation configuration

demo03c
recursion-limit

SpiralTrain developer courses

GraphRecursionError

- The recursion limit bounds graph steps in one invocation
- A node with an edge back to itself raises `GraphRecursionError` at the limit
- A higher limit permits more steps before the runtime stops the graph
- Every cycle needs a routing condition that exits, such as a cap on attempts

SpiralTrain developer courses

Command

- `Command` combines a state update with a routing decision :

```
from langgraph.types import Command

def triage(state):
    return Command(update={"assigned": "urgent"}, goto="page")
```

- `update` changes state from inside a node
- `goto` selects one or more next nodes
- A node returning `Command` routes itself ; it needs no conditional edge

demo04
command-and-send

SpiralTrain developer courses

Parallel Branches and Send

- Nodes with a shared predecessor can run in parallel
- `Send` creates runtime branches with individual inputs :

```python
from langgraph.types import Send

def fan_out(state):
    return [Send("report", {"depot": name}) for name in state["depots"]]
```

- A `collect` node summarises the reports as `3 depots checked`
- Map-reduce combines fan-out work with a reduction step

demo04b
send-map-reduce

SpiralTrain developer courses

Deferred Nodes

- A deferred node waits until the other scheduled work is complete

```
builder.add_node("collect", collect, defer=True)
```

- A deferred `collect` waits for every depot report before it summarises
- `defer` is available in LangGraph 1.2.15

demo04c
deferred-nodes

SpiralTrain developer courses

Compiling a Graph

- `compile` checks the builder's structure and returns a runnable graph
- A checkpointer is attached at compile time
- Runtime options such as `recursion_limit` go into each invocation
- A builder is compiled once ; the graph is invoked many times

SpiralTrain developer courses

Visualising a Graph

- A compiled graph returns a Mermaid description :

```
print(graph.get_graph().draw_mermaid())
```

- Mermaid text can be displayed in Markdown or a compatible viewer
- Mermaid text rendering requires no network service

SpiralTrain developer courses

Functional API

- The Functional API expresses a workflow with decorated functions :

```python
from langgraph.func import entrypoint, task

@task
def clean(text: str) -> str:
    return text.strip()

@entrypoint()
def workflow(text: str) -> str:
    return clean(text).result()
```

- Tasks can run in parallel and return values to the entrypoint

demo05b
functional-api

SpiralTrain developer courses

Graph API or Functional API

- **Graph API :**
  - Explicit nodes and edges are drawable and inspectable
  - Branches and loops appear directly in the graph definition
- **Functional API :**
  - Ordinary Python control flow expresses the sequence
  - Tasks mark work that can run and resume independently
- **Both APIs use the LangGraph runtime**
- **A task can call a compiled graph**

SpiralTrain developer courses

A Model in a Node

- **A model call is one node among plain functions :**
  - `classify` asks the model whether a work order is electrical or mechanical
  - A routing function sends it to the electrical or the mechanical bay
- **Model output can vary between runs**
- **The graph fixes every step around that one judgment**

demo06
a-model-in-a-node

SpiralTrain developer courses

Summary : Graph Fundamentals

- Directed graphs describe steps and allowed transitions
- State machines combine state with transitions
- `StateGraph` connects nodes with fixed and conditional edges
- `START`, `END`, `Command`, and `Send` express graph control flow
- Cycles run under a configurable recursion limit
- Deferred nodes wait for scheduled work to finish
- Compilation prepares a runnable graph that can be visualised
- Graph API and Functional API share the LangGraph runtime

SpiralTrain developer courses

- Introduction to LangGraph
- Graph Fundamentals
- **State Management**
- Building Agents
- Multi-Agent Patterns
- Agent Communication
- Advanced Workflows
- Deep Agents
- Production Deployment
- Real-World Applications

- TypedDict State
- Dataclass State
- Pydantic State
- Input and Output Schemas
- Private State
- Reducers
- Merging State Updates
- MessagesState
- add_messages
- Custom Reducers
- Overwriting a Reduced Key
- Concurrent Updates
- Checkpointers and Persistence
- Threads
- SqliteSaver
- PostgreSQL Checkpoint Storage
- Inspecting Thread State
- State History
- Restoring Saved State
- Time Travel
- Forking a Checkpoint
- DeltaChannel
- Long-Term Memory
- Streaming Modes
- Debugging State
- The v2 Stream Format
- v3 Event Projections

TypedDict State

- A state schema names the keys shared between nodes
- `TypedDict` describes dictionary fields for type checking
- Nodes return partial updates ; omitted keys retain their values

```
class TicketState(TypedDict):
    ticket_id: str
    symptom: str
    status: str

def mark_open(state: TicketState):
    return {"status": "open"}
```

- `TypedDict` annotations are not checked at runtime ; a wrong type passes

demo01
typed-dict

Dataclass State

- A dataclass supplies defaults for fields omitted from input
- Nodes receive a dataclass instance and return dictionary updates

```
@dataclass
class TicketDefaults:
    ticket_id: str
    status: str = "new"

def inspect_default(state: TicketDefaults):
    return {"status": "open"}
```

- Attribute access uses `state.status`
- Dataclass annotations do not enforce field types

Pydantic State

- Pydantic validates state when it is constructed for a node
- Field constraints reject invalid input before the node runs

```
class ValidatedTicket(BaseModel):
    ticket_id: str
    hours: float = Field(ge=0)

def round_hours(state: ValidatedTicket):
    return {"hours": round(state.hours, 1)}
```

- A node that returns `hours=-1` passes unchecked ; only node input is validated
- Validation adds runtime work compared with `TypedDict`

demo01b
pydantic

Input and Output Schemas

- The internal schema contains the graph's working state
- `input_schema` selects the fields accepted at entry
- `output_schema` selects the fields returned by `invoke`

```
public = StateGraph(
    InternalState,
    input_schema=TicketInput,
    output_schema=StatusOutput,
)
```

- A ticket id enters ; only the resulting status leaves
- The filtered keys remain in the checkpoints

demo01c
input-and-output

SpiralTrain developer courses

Private State

- Nodes can declare additional channels through their input schemas
- A private schema carries internal communication between nodes

```
class PrivateState(TypedDict):
    working_note: str

def finish(state: PrivateState):
    return {"status": state["working_note"] + " at depot"}
```

- Private channels can be excluded from the graph's public output
- `working_note` still appears in checkpoints and in the `values` stream
- Hiding a key from the output gives no access control

demo01d
private-state

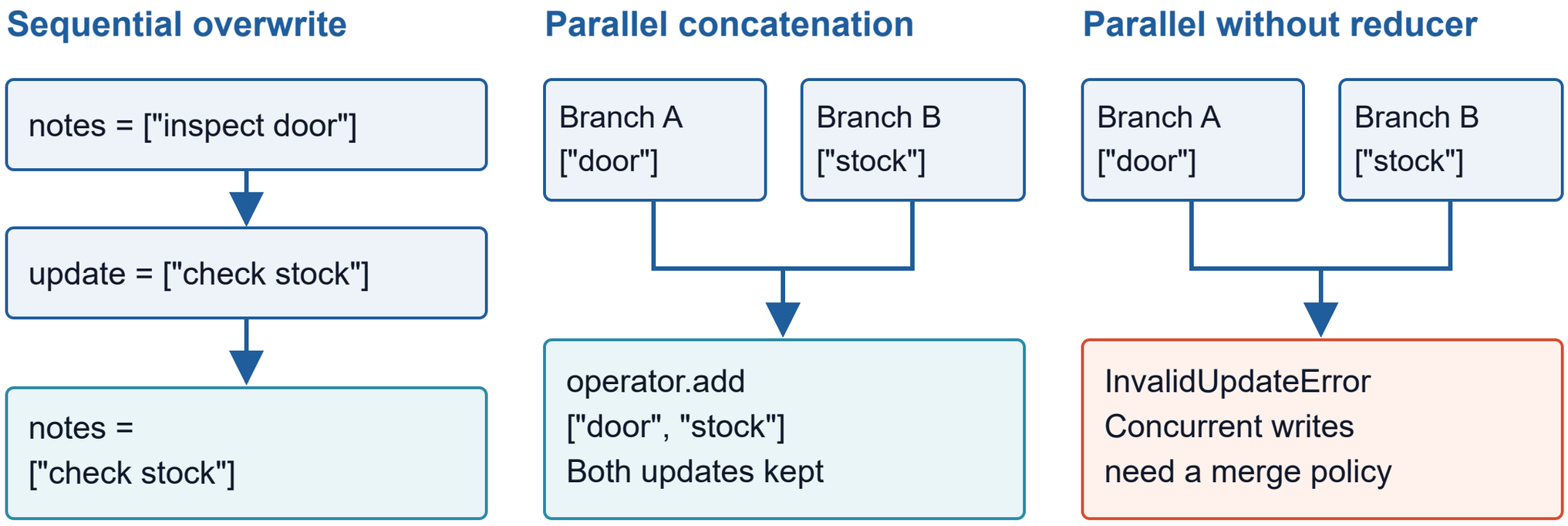
Reducers

- A reducer is the merge policy for one key of the state
- Without a reducer, a sequential update overwrites the old value
- `Annotated` attaches a reducer to a field

```
class Collect(TypedDict):
    collected: Annotated[list[str], operator.add]
    last: str
```

- `operator.add` concatenates the incoming list with the saved list
- Nodes return new entries for reduced keys
- Returning the accumulated list again can duplicate entries

Merging State Updates



Illustrative state values ; concatenation order is not a business priority

- Each state key has its own reducer, or none
- A reducer defines how updates to that key combine
- Two workers that disagree both land in the list ; choosing one needs a judge or a review

MessagesState

- `MessagesState` supplies a `messages` field with `add_messages`
- Additional fields extend the schema through inheritance

```python
from langgraph.graph import MessagesState

class ServiceState(MessagesState):
    depot: str
```

- A node can return only its new assistant message
- The reducer preserves earlier messages
- The depot field keeps the default overwrite policy

SpiralTrain developer courses

add_messages

- `add_messages` manages message identities :
  - New ids append messages
  - Existing ids replace the corresponding entries
  - `RemoveMessage(id=...)` removes a matching entry

```
history = add_messages(history, [
    AIMessage("Assigned to workshop", id="reply")
])
history = add_messages(history, [RemoveMessage(id="request")])
```

- Message dictionaries and role-content tuples become message objects
- Stable ids allow an application to edit an earlier message

demo02
add-messages

Custom Reducers

- A custom reducer receives the current value and an incoming update
- The return value becomes the next value of that state key

```python
def merge_findings(current: list[str], incoming: list[str]):
    return sorted(set(current) | set(incoming))

class Findings(TypedDict):
    findings: Annotated[list[str], merge_findings]
```

- Set union removes duplicate findings in this example
- Sorting gives a stable presentation order
- Reducers should avoid mutating their arguments

demo02b
custom-reducers

SpiralTrain developer courses

Overwriting a Reduced Key

- An ordinary update still goes through the reducer
- An empty incoming list does not clear a concatenated list
- `Overwrite` bypasses the reducer for a replacement

```
from langgraph.types import Overwrite

reset.add_node("clear", lambda state: {
    "findings": Overwrite(value=[])
})
```

- The replacement can be a new value or the field's empty value
- Multiple overwrites of the same key in one step are invalid

demo02c
overwriting

SpiralTrain developer courses

Concurrent Updates

- Parallel nodes read the state available at the start of a superstep
- Their updates merge after that step's nodes complete
- Two writes to one unreduced key raise `InvalidUpdateError`

```
parallel.add_edge(START, "parts")
parallel.add_edge(START, "workshop")
```

- A reducer lets both branches add their workshop findings to one key
- Separate keys suit branches that produce different results
- A reducer keeps both findings ; picking the correct one is a separate step

demo02d
concurrent-updates

Checkpointers and Persistence

- A checkpointer saves execution state at superstep boundaries :
  - State values and the nodes scheduled next
  - Metadata and the config that identifies the checkpoint
- Checkpoints support memory between invocations and saved pauses
- Persistent storage allows restoration after a process restart

```python
from langgraph.checkpoint.memory import InMemorySaver

graph = builder.compile(checkpointer=InMemorySaver())
```

- `InMemorySaver` loses its checkpoints when the process ends

Threads

- `thread_id` identifies one sequence of saved graph states
- Reusing the id loads that thread's state before applying new input
- A different id starts a separate history

```
morning = {"configurable": {"thread_id": "u1-morning"}}
graph.invoke({"messages": [("user", "My bus is 4012.")]}, morning)
graph.invoke({"messages": [("user", "Which bus?")]}, morning)
```

- Messages accumulate through their reducer across invocations
- The same thread recalls bus 4012 and depot Noord ; a new thread does not
- A thread id is not authentication ; the application decides who may read it
- One user can own many threads, such as `u1-morning`

demo03
threads

SqliteSaver

- `langgraph-checkpoint-sqlite` supplies `SqliteSaver`
- A file-backed saver keeps checkpoints after the connection closes
- The context manager owns the connection lifetime

```
from langgraph.checkpoint.sqlite import SqliteSaver

with SqliteSaver.from_conn_string(str(database)) as saver:
    persistent = builder.compile(checkpointer=saver)
    persistent.invoke(inputs, config)
```

- Graph calls that use the saver belong inside the `with` block
- SQLite suits local development and small applications

demo03b
sqlite-saver

PostgreSQL Checkpoint Storage

- `langgraph-checkpoint-postgres` provides a PostgreSQL backend
- Several application processes can share one PostgreSQL checkpoint store
- Production operation includes :
  - Connection management and schema setup
  - Backups and checkpoint retention
  - Access controls for thread data
- The checkpointer persists execution state within a thread
- Cross-thread application memory uses a Store

Inspecting Thread State

- `get_state` returns a `StateSnapshot` for the selected checkpoint

```
snapshot = graph.get_state(morning)
print(snapshot.values)
print(snapshot.next)
print(snapshot.config["configurable"]["checkpoint_id"])
```

- `values` contains the saved channel values
- `next` names nodes scheduled to run from that checkpoint
- An empty `next` indicates no scheduled continuation
- A config with `checkpoint_id` selects a historical snapshot

State History

- `get_state_history` yields saved checkpoints, newest first
- Each snapshot keeps its own config and parent relationship

```
history = list(graph.get_state_history(config))
for snapshot in history:
    print(snapshot.metadata["step"], snapshot.next)

before_record = next(
    snapshot for snapshot in history
    if snapshot.next == ("record",)
)
```

- A snapshot's config selects the point for replay or forking

demo04
state-history

SpiralTrain developer courses

Restoring Saved State

- A new graph instance can load checkpoints from the same backend
- Restoration requires the same thread id and compatible graph code
- `invoke(None, config)` continues from the saved scheduled nodes

```
with SqliteSaver.from_conn_string(str(database)) as saver:
    restored = builder.compile(checkpointer=saver)
    print(restored.get_state(config).next)
    restored.invoke(None, config)
```

- Supplying new input applies a new input update
- The demo closes the connection, compiles a new graph and runs `record` from the saved pause

Time Travel

- Replay continues execution from a historical checkpoint
- Nodes before the checkpoint retain their saved results
- Nodes scheduled after it execute again

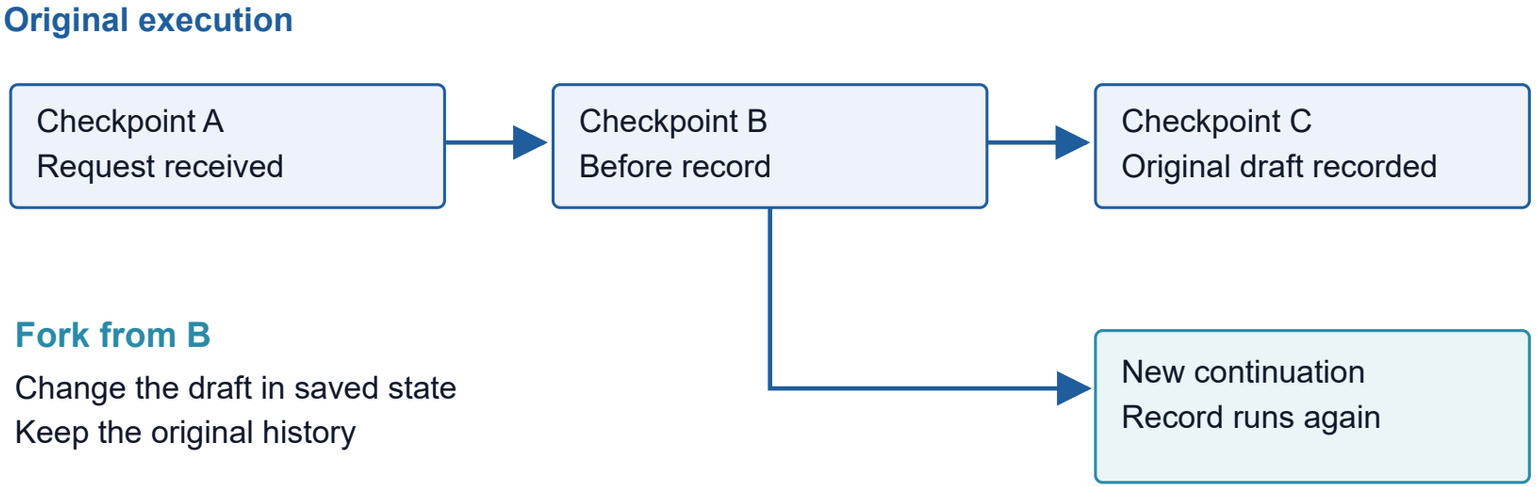
```
replayed = graph.invoke(None, before_draft.config)
```

- Replaying from before `draft` calls the model again ; the wording can change
- A replayed part order can be placed a second time
- A checkpoint with no scheduled nodes has no continuation to replay
- Reading state with `get_state` does not replay execution

demo04b
replay

Forking a Checkpoint

- `update_state` writes a new checkpoint from a saved one
- Updates pass through the same reducers as node updates
- `as_node` attributes the update and determines the continuation



The fork reuses a checkpoint ; it is not a rollback of external actions

- The original checkpoint remains addressable
- A fork does not cancel a part order the earlier run already placed

demo04c
forking

SpiralTrain developer courses

DeltaChannel

- `DeltaChannel` is beta in LangGraph 1.2.15
- Saved writes reconstruct a growing value between periodic snapshots
- A delta reducer receives the accumulated value and a batch of writes

```
def merge_batches(current, writes):
    return current + [item for batch in writes for item in batch]

class LongThread(TypedDict):
    entries: Annotated[list[str], DeltaChannel(merge_batches)]
```

- Reducers must be deterministic and invariant to batching
- Replay and snapshot storage depend on checkpointer support
- Delta storage does not shorten the model's context

Long-Term Memory

- A Store keeps application data across threads
- Namespace tuples group documents ; keys identify individual documents

```
store = InMemoryStore()
namespace = ("users", "u1", "profile")
store.put(namespace, "depot", {"text": "depot: Noord"})
print(store.get(namespace, "depot").value)
```

- `put` with the same namespace and key replaces the document
- `search` retrieves items within a namespace prefix
- `delete` removes a document
- `runtime.store` and trusted runtime context scope a node's access

demo05
the-store

Streaming Modes

- `stream_mode` chooses what a run emits :
  - `values` emits accumulated state after each step
  - `updates` emits node names and the updates they returned
  - `messages` emits model message chunks with metadata
  - `custom` emits application data through `get_stream_writer`
- Streaming executes the graph while the caller consumes its output
- Inspecting an existing checkpoint uses `get_state`

```
for part in graph.stream(inputs, config,
                         stream_mode=["values", "updates"], version="v2"):
    print(part["type"], part["data"])
```

demo06
stream-modes

Debugging State

- `debug` combines checkpoint and task events :
  - Checkpoint values and scheduled work
  - Task starts and completions, including results and errors
- `checkpoints` and `tasks` select those event families separately
- Checkpoint events require a checkpointer
- `updates` locates the node that wrote a value
- `values` shows the result after reducers merged updates
- Studio provides a visual graph and state inspection interface

The v2 Stream Format

- `stream(..., version="v2")` emits a common dictionary format
- `type` selects the stream mode ; `data` holds its payload
- `ns` identifies the graph namespace

```
for part in graph.stream(inputs, config,
                         stream_mode=["custom", "updates"], version="v2"):
    print(part["type"], part["ns"], part["data"])
```

- Root graph events use an empty namespace tuple
- Values events also carry interrupt information
- The default v1 format varies with modes and subgraph settings

SpiralTrain developer courses

v3 Event Projections

- `stream_events(..., version="v3")` returns a `GraphRunStream`
- The API is experimental on the pinned stack
- Projections select typed views of one run :
  - `values` yields state snapshots
  - `messages`, `lifecycle` and `subgraphs` expose other views

```
run = small_graph.stream_events({"value": 0}, version="v3")
for state in run.values:
    print(state)
print(run.output)
```

- Iteration drives the synchronous run without a background thread
- v3 owns stream modes and subgraph streaming settings

SpiralTrain developer courses

Summary : State Management

- Schemas define state fields and public graph interfaces
- Reducers define how updates to each state key combine
- Parallel writes need a merge policy or separate keys
- Checkpointers retain thread state and scheduled continuation
- Saved checkpoint configs select history for replay and forking
- A Store keeps application memory across threads
- Stream modes expose node updates and accumulated state
- DeltaChannel is beta and v3 event streaming experimental on the pinned stack

SpiralTrain developer courses

- Introduction to LangGraph
- Graph Fundamentals
- State Management
- **Building Agents**
- Multi-Agent Patterns
- Agent Communication
- Advanced Workflows
- Deep Agents
- Production Deployment
- Real-World Applications

- Tool-Calling Agents
- Models and Messages
- Defining Tools
- Agent Single Pass
- Agent Loop
- ToolNode
- tools_condition
- ReAct Agent
- create_agent
- An Agent as a Node
- Custom Agent Logic
- Custom Agent State
- Middleware Hooks
- Writing Logging Middleware
- Fault Tolerance
- Node Run Timeout
- Node Idle Timeout
- Node Error Handler
- Tool Errors
- Model Fallback
- Model Call Limit
- Tool Call Limit
- LangSmith Tracing
- Testing Strategies
- Unit Tests with a Fake Model
- Testing Memory
- Running pytest
- Best Practices

Tool-Calling Agents

- An agent combines a chat model, tools and an execution loop
- Each model request carries the conversation and the available tool schemas :
  - The model selects a tool and its arguments, such as `bus_info` for CE12-2021-0117
  - Application code executes the requested operation
- Tool results become messages in the conversation
- A model response without tool calls normally ends the loop
- Limits, interrupts and errors can also stop execution

SpiralTrain developer courses

Models and Messages

- A chat model accepts a sequence of messages and returns an `AIMessage`
- Message roles identify the source and purpose :
  - `SystemMessage` supplies instructions
  - `HumanMessage` supplies the request
  - `AIMessage` contains model output and optional `tool_calls`
  - `ToolMessage` contains a result linked by `tool_call_id`
- `llm.bind_tools(tools)` exposes schemas to the model
- Binding tools does not execute them

```
bound = llm.bind_tools(tools)
message = bound.invoke([("human", "Check the bus parts.")])
print(message.content, message.tool_calls)
```

SpiralTrain developer courses

Defining Tools

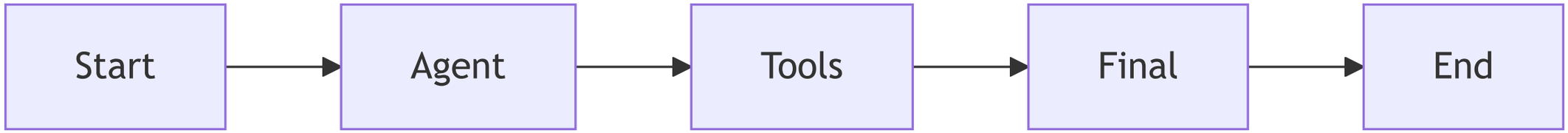
- `@tool` builds a schema from a function :
  - The name identifies the operation
  - Type hints describe the arguments
  - The docstring describes when to use it
- A tool validates its arguments before it touches the database
- Parameterised SQL keeps a fleet number from being read as SQL

```
@tool
def bus_info(fleet_no: str) -> str:
    """Look up a Veenhaven bus model and depot by its fleet number."""
    with sqlite3.connect(DB) as conn:
        row = conn.execute(
            "SELECT model, depot_id FROM buses WHERE fleet_no = ?",
            (fleet_no,)).fetchone()
    return json.dumps(dict(zip(["model", "depot_id"], row))) if row else "Unknown bus"
```

SpiralTrain developer courses

Agent Single Pass

- The model chooses tools during a single turn
- A final node receives the tool results and writes an answer
- Dependent calls need results from an earlier call :
  - `bus_info` supplies the bus model
  - `parts_for_model` needs that model identifier
- A single pass leaves `parts_for_model` without a real model id ; the model skips it or guesses one
- The saved run reports the parts lookup as missing



SpiralTrain developer courses

Agent Loop

- Each tool result gives the model another turn
- The model reads the result before selecting the next operation
- The graph routes by the presence of tool calls :
  - Calls present : execute the tools
  - Calls absent : end the graph
- The saved loop finds CE-DA-03, out of stock, in 3 model calls

```
loop.add_node("agent", call_model)
loop.add_node("tools", run_tools)
loop.add_edge(START, "agent")
loop.add_conditional_edges("agent", route)
loop.add_edge("tools", "agent")
result = loop.compile().invoke(question, {"recursion_limit": 16})
```

demo01
react-as-a-graph

ToolNode

- `ToolNode` executes tool calls from the last message :
  - Matches each call to a registered tool
  - Validates the arguments before execution
  - Returns a result with the corresponding call identifier
- Several calls can execute in parallel
- Tools can return `Command` updates for custom state and routing
- An exception raised inside a tool needs its own handling policy

```python
from langgraph.prebuilt import ToolNode

builder.add_node("tools", ToolNode(tools))
```

SpiralTrain developer courses

tools_condition

- `tools_condition` reads the last message in `state["messages"]` :
  - Tool calls present : returns `"tools"`
  - No tool calls : returns `"__end__"`
- The default route expects a node named `tools`
- A path map can translate the route to another node name

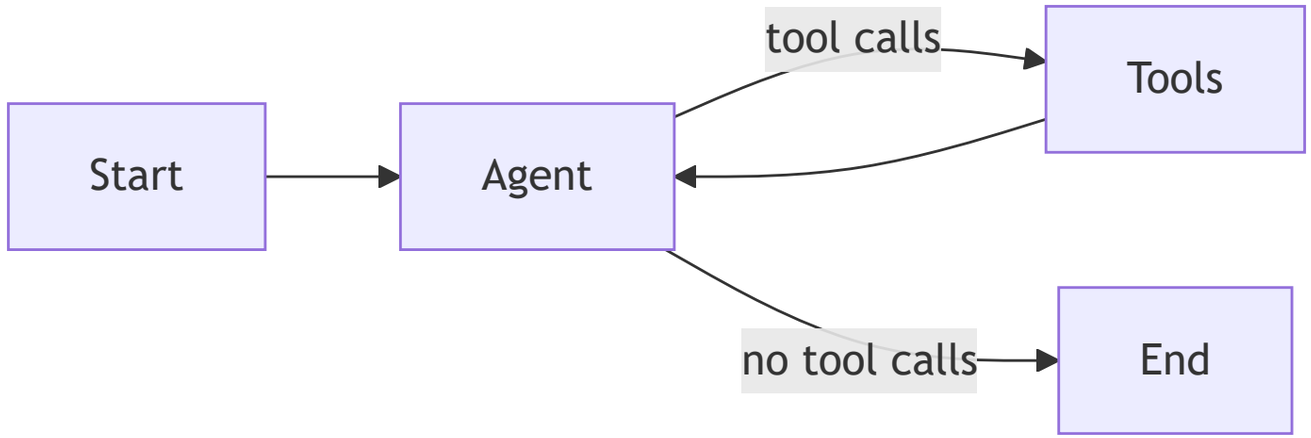
```python
from langgraph.prebuilt import tools_condition

builder.add_conditional_edges("agent", tools_condition)
builder.add_edge("tools", "agent")
graph = builder.compile()
```

demo01b
toolnode-and-tools-condition

ReAct Agent

- **ReAct interleaves reasoning, actions and observations**
- **Tool-calling agents use the same interaction pattern :**
  - The model selects an operation
  - A tool supplies an observation
  - The model uses the observation in its next decision
- **The graph shows each action and observation without a printed reasoning transcript**
- **A custom node can enforce an order, such as the bus lookup before the parts lookup**



SpiralTrain developer courses

create_agent

- `create_agent` builds and compiles an agent graph :
  - A model node selects actions or produces an answer
  - A tools node executes requested operations
  - Middleware adds hooks around the loop
- Invocation returns the final state, including the message history

```
from langchain.agents import create_agent

agent = create_agent(llm, tools=tools,
    system_prompt="Look up the bus first. Use the database for stock. Report unknown records.")
result = agent.invoke(question, {"recursion_limit": 16})
print(result["messages"][-1].content)
print(list(agent.get_graph().nodes))
```

demo02
create-agent

SpiralTrain developer courses

An Agent as a Node

- A compiled agent can be a node in a parent graph
- Keys the parent and child share, such as `messages`, pass in and out
- The parent controls the surrounding workflow :
  - Deterministic preprocessing before the agent
  - A required handover after its answer
- Different state shapes need a wrapper that maps inputs and outputs

```
parent = StateGraph(VisitState)
parent.add_node("parts_agent", agent)
parent.add_node("handover", handover)
parent.add_edge(START, "parts_agent")
parent.add_edge("parts_agent", "handover")
parent.add_edge("handover", END)
```

demo02b
agent-as-a-node

SpiralTrain developer courses

Custom Agent Logic

- A custom model node controls its prompt and the messages it sends
- Graph edges enforce required steps outside model choice
- A prompt describes the goal and the permitted operations :
  - Look up the bus before selecting compatible parts
  - Report missing records and unavailable stock
  - Refer safety decisions to the workshop
- A prompt guides the model ; only code enforces an action boundary

```
def call_model(state: MessagesState):
    return {"messages": [bound.invoke([prompt, *state["messages"]])]}
```

SpiralTrain developer courses

Custom Agent State

- `AgentState` contains message history and supports additional fields
- State fields belong to the running conversation :
  - The depot the parts search is limited to
  - A count of model calls, updated by middleware
- Middleware reads and updates those fields
- `ToolRuntime` injects state into tools without exposing a runtime argument to the model

```
class DepotState(AgentState):
    depot: str
    model_calls: NotRequired[int]

depot_agent = create_agent(llm, tools=[depot_parts],
    state_schema=DepotState, middleware=[depot_prompt, count_calls])
```

demo02c
custom-agent-state

SpiralTrain developer courses

Middleware Hooks

- Agent middleware extends the loop through named hooks

hook	execution point
`before_agent`	before the agent loop
`before_model`	before each model call
`after_model`	after each model call
`after_agent`	after the agent loop
`wrap_model_call`	around a model request
`wrap_tool_call`	around a tool request

- State hooks can return updates
- Wrappers can call the handler, change the request, or return an alternative result
- Async invocation uses the corresponding async hooks

SpiralTrain developer courses

Writing Logging Middleware

- A middleware class can observe every model turn
- A decorated wrapper can observe tool arguments and results
- The saved log shows the bus lookup, then the parts lookup
- A log records tool names and arguments, never keys or personal data

```
class Logger(AgentMiddleware):
    def before_model(self, state, runtime):
        print("model input messages:", len(state["messages"]))

@wrap_tool_call
def log_tool(request, handler):
    print("tool:", request.tool_call["name"], request.tool_call["args"])
    result = handler(request)
    print("result:", result.content)
    return result
```

demo03
middleware-hooks

SpiralTrain developer courses

Fault Tolerance

- A retry policy repeats selected failed node attempts
- `max_attempts` includes the initial attempt
- `retry_on` selects retryable exceptions
- Node retries precede the node error handler
- The demo node fails twice and succeeds on attempt 3
- A retried part order must not place the order twice

```
from langgraph.types import RetryPolicy

builder.add_node("inventory", inventory, retry_policy=RetryPolicy(
    max_attempts=3, initial_interval=0.01,
    jitter=False, retry_on=ConnectionError))
```

demo04
node-failures

SpiralTrain developer courses

Node Run Timeout

- Node `timeout` bounds one async attempt
- A number sets a run timeout in seconds
- `TimeoutPolicy.run_timeout` caps the attempt's wall-clock duration
- Progress does not refresh the run limit
- Expiry raises `NodeTimeoutError` and discards pending writes from the attempt
- Synchronous nodes with a timeout are rejected at graph compilation

```
await timed(slow_inventory, TimeoutPolicy(run_timeout=0.08))
```

SpiralTrain developer courses

Node Idle Timeout

- `idle_timeout` bounds the time without observable progress
- The refresh policy selects progress signals :
  - `auto` accepts graph and LangChain callback progress
  - `heartbeat` accepts explicit `runtime.heartbeat()` calls
- A heartbeat refreshes the idle limit without changing the run limit
- With both limits set, the first limit reached cancels the attempt

```
await timed(progressing_inventory,
    TimeoutPolicy(run_timeout=0.08, idle_timeout=0.08,
                  refresh_on="heartbeat"))
```

demo04b
node-timeout

SpiralTrain developer courses

Node Error Handler

- `error_handler` runs after retries are exhausted or no retry applies
- The handler receives state and a typed `NodeError` parameter :
  - `error.node` identifies the failed node
  - `error.error` contains the exception
- A returned `Command` updates state and routes recovery
- An error raised by the handler fails the run

```
def on_error(state: LookupState, error: NodeError) -> Command:
    return Command(update={"status": "manual stock check required"},
                   goto="notify")

builder.add_node("inventory", unavailable,
    error_handler=on_error, retry_policy=RetryPolicy(max_attempts=2,
        retry_on=ConnectionError, initial_interval=0.01, jitter=False))
```

demo04c
error-handler

SpiralTrain developer courses

Tool Errors

- An unhandled tool execution exception stops the agent
- `ToolErrorMiddleware` selectively converts execution failures into error `ToolMessage` results
- The error callback decides whether to handle the failure :
  - Returned content becomes an error message for the model
  - `None` lets the exception propagate
- Tool argument validation is handled upstream by `ToolNode`

```
def on_error(exc, request):
    if isinstance(exc, TimeoutError):
        return "Stock service unavailable. Do not infer stock. Ask for a manual stock check."
    return None

safe = create_agent(llm, tools=[live_stock],
                    middleware=[ToolErrorMiddleware(on_error)])
```

demo05
agent-failures

SpiralTrain developer courses

Model Fallback

- `ModelFallbackMiddleware` tries fallback models when a model call raises
- The primary model is supplied to `create_agent`
- Fallbacks are supplied to the middleware in order
- If every attempt fails, the last exception propagates
- A fallback model needs tool calling and the agent's request settings

```
agent = create_agent(primary, tools=tools,
    middleware=[ModelFallbackMiddleware(llm)],
    system_prompt="Use the database for stock. Look up the bus before its parts.")
out = agent.invoke(question, {"recursion_limit": 16})
```

demo05b
model-fallback

SpiralTrain developer courses

Model Call Limit

- `ModelCallLimitMiddleware` bounds model calls :
  - `run_limit` applies to the current invocation
  - `thread_limit` accumulates in persisted state
- `exit_behavior="end"` ends with a limit message
- `exit_behavior="error"` raises an exception
- Graph recursion limits count graph steps, including middleware nodes

```
middleware = ModelCallLimitMiddleware(run_limit=1, exit_behavior="end")
agent = create_agent(repeated_stock(), tools=[bus_info],
                     middleware=[middleware])
```

SpiralTrain developer courses

Tool Call Limit

- `ToolCallLimitMiddleware` counts tool calls per run or thread
- `tool_name` restricts the counter to one tool
- Exit behaviour controls what happens at the limit :
  - `continue` returns an error for blocked calls and keeps the agent running
  - `end` stops the agent
  - `error` raises an exception
- With `continue`, an agent can keep requesting the blocked tool ; a model-call limit stops it

```
middleware = ToolCallLimitMiddleware(tool_name="bus_info",
                                     run_limit=1, exit_behavior="end")
```

demo05c
call-limits

SpiralTrain developer courses

LangSmith Tracing

- **Tracing records the run tree with model and tool calls :**
  - Inputs, outputs and errors
  - Timing and provider-reported token usage
- **`tracing_context` can enable tracing for a scoped block**
- **The client uses the configured LangSmith region endpoint**
- **Project names group related traces**
- **A tracing failure is printed and the answer still returns**

```
with tracing_context(enabled=True, client=client, project_name=PROJECT):
    out = agent.invoke(question)
client.flush()
```

demo06
tracing-with-langsmith

SpiralTrain developer courses

Testing Strategies

- Unit tests check tools and middleware without a model API
- Scripted agent tests check graph execution and tool-result handling
- Real-model evaluations check choices and answer quality on labelled cases
- Production monitoring detects failures outside the evaluation set
- Each layer answers a different question :
  - A scripted tool call proves the wiring executes that call
  - An evaluation checks whether the real model chooses a suitable call
- Incidents supply new regression and evaluation cases

SpiralTrain developer courses

Unit Tests with a Fake Model

- `GenericFakeChatModel` returns scripted messages
- Agents with tools need a `bind_tools` override on the fake
- Scripted tool calls exercise the real tool and agent wiring
- A scripted final answer does not prove the model interpreted the result

```
class FakeToolModel(GenericFakeChatModel):
    def bind_tools(self, tools, **kwargs):
        return self

fake = FakeToolModel(messages=iter([
    call("CE-DA-03"), AIMessage("CE-DA-03 is out of stock.")]))
```

SpiralTrain developer courses

Testing Memory

- `InMemorySaver` persists graph state within a process
- A thread identifier selects the conversation
- Tests check memory independently of scripted answer text :
  - Earlier tool results remain in the saved message history
  - A different thread starts without that history
- A new agent and saver per test isolate cases
- Persistent production storage needs additional integration tests

```
config = {"configurable": {"thread_id": "first"}}
saved = agent.get_state(config).values["messages"]
assert any(m.type == "tool" and m.content == "0" for m in saved)
```

SpiralTrain developer courses

Running pytest

- A test file can run independently of the notebook
- Assertions should check observable behaviour :
  - Tool result content and call identifier
  - Explicit unknown-record handling
  - Saved history and thread isolation
- `check_returncode` fails the cell when a test fails

```
run = subprocess.run([sys.executable, "-B", "-m", "pytest", "-p",
                      "no:cacheprovider", "-q", str(test_path)],
                     capture_output=True, text=True)
print(run.stdout)
run.check_returncode()
```

demo07b
pytest

SpiralTrain developer courses

Best Practices

- A single agent with a narrow tool list comes first
- Required steps sit in graph edges, outside the model's choice
- Tools return `Unknown part` or a recorded zero, so the model has no gap to fill
- Limits bound model calls, tool calls and external-service time
- Retries suit transient failures of idempotent operations
- Traces record the tool path ; measurements record calls and tokens
- Scripted models test the wiring ; real-model evaluations test quality
- Repair and safety decisions go to the workshop ; a stock lookup does not clear a bus for service

SpiralTrain developer courses

Summary : Building Agents

- A tool-calling agent is a model and tools connected by a loop
- `ToolNode` and `tools_condition` provide dispatch and routing
- `create_agent` compiles the loop into a graph that can be a node
- Custom state and middleware extend agent behaviour
- Retry policies, timeouts and error handlers control node failures
- Agent middleware handles tool errors, model fallback and call limits
- Traces expose execution, while scripted tests verify wiring and memory
- Real-model evaluations are required to assess model choices and answer quality

- Introduction to LangGraph
- Graph Fundamentals
- State Management
- Building Agents
- **Multi-Agent Patterns**
- Agent Communication
- Advanced Workflows
- Deep Agents
- Production Deployment
- Real-World Applications

- Multi-Agent Concepts
- Agent Roles and Responsibilities
- Choosing a Pattern
- Pattern Capabilities
- Documentation Call Counts
- Subagents
- Subagent Context
- Manager-Worker
- A Supervisor Node
- Hierarchical Teams
- Router
- Router Code
- Delegation and Handoff
- Handoffs
- Fixed Handoffs
- Handoff Middleware
- Skills
- Specialized Agents
- Custom Workflow
- Sequential Pattern
- Parallel Agents
- Orchestration in Code
- Collaborative Agents
- Maker-Checker
- Checkers Written in Code
- Debate with a Judge
- Generate and Filter
- Orchestration Shapes
- Evaluating the Boundaries
- Supervisor and Swarm Packages
- The Earlier Pattern Probe

SpiralTrain developer courses

Multi-Agent Concepts

- **A multi-agent system splits work across agents with separate context**
- **Multi-agent design is context engineering :**
  - Decide which facts, instructions and tools each agent sees
  - Decide which findings return to the caller
- **Measurable problems justify another agent :**
  - Tool overload causes incorrect tool selection
  - Specialised prompts compete inside one context
  - Intermediate work crowds the context used for the answer
- **One agent comes first, measured on quality, calls, tokens and seconds**
- **Each pattern is built from LangGraph nodes before a harness hides it**

Agent Roles and Responsibilities

- **Each agent gets one responsibility, stated in its prompt :**
  - A parts specialist answers stock questions from the database
  - A manuals specialist returns error guidance with a source
  - A coordinator delegates and writes the final answer
- **Each agent gets the tools needed for its role**
- **A description tells the coordinator when to use each specialist**
- **A delegated task includes relevant identifiers and completion criteria**
- **A missing fact is traced in the worker's answer first, then in the synthesis**
- **A different model per role needs an evaluation that shows the gain**

SpiralTrain developer courses

Choosing a Pattern

- **The LangChain documentation names these patterns :**

Pattern	Control	Suitable task
Subagents	Main agent calls workers as tools	Isolated specialist work
Handoffs	Tool updates the active stage	Multi-stage conversation
Skills	Agent loads instructions	Specialities in one context
Router	Classifier dispatches and combines	Distinct input categories
Custom workflow	Graph fixes the execution flow	Business process with rules

- **Patterns nest ; a workflow node can be a router or a subagent**
- **A parts subagent and a parts skill route the same way ; only the subagent gets its own context**

SpiralTrain developer courses

Pattern Capabilities

- LangChain's documentation rates the patterns by capability :

Pattern	Separate teams	Parallel	Multi-hop	User interaction
Subagents	High	High	High	Low
Handoffs	Unrated	Unrated	High	High
Skills	High	Medium	High	High
Router	Medium	High	Unrated	Medium

- The ratings are LangChain's, for the documented architectures
- Custom workflows combine capabilities through explicit graph structure
- A rating says nothing about answer quality or cost on the bus question

Documentation Call Counts

- LangChain illustrates these calls and processed tokens :

Pattern	One request	Repeat turn	Three domains
Subagents	4 calls	4 calls	5 calls, about 9K tokens
Handoffs	3 calls	2 calls	7+ calls, about 14K+ tokens
Skills	3 calls	2 calls	3 calls, about 15K tokens
Router	3 calls	3 calls	5 calls, about 9K tokens

- The repeat turn reuses the active agent or loaded skill
- Loaded instructions remain in subsequent calls to the same agent
- The counts come from LangChain's own examples ; the bus measurements are in demo09

Subagents

- A subagent is an agent wrapped as a tool
- Each worker's model and tool loop is a LangGraph graph

```
workers = specialists(llm)

@tool
def call_parts(question: str) -> str:
    """Ask the parts specialist about stock and location."""
    result = workers["parts"].invoke(ask(question), LIMIT)
    return last_text(result)

coordinator = agent_by_hand(llm, [call_parts, call_manuals],
    "Delegate both data questions, then answer briefly.")
```

- The coordinator sees the worker's final answer as a tool result, never its tool calls

demo01
subagents

SpiralTrain developer courses

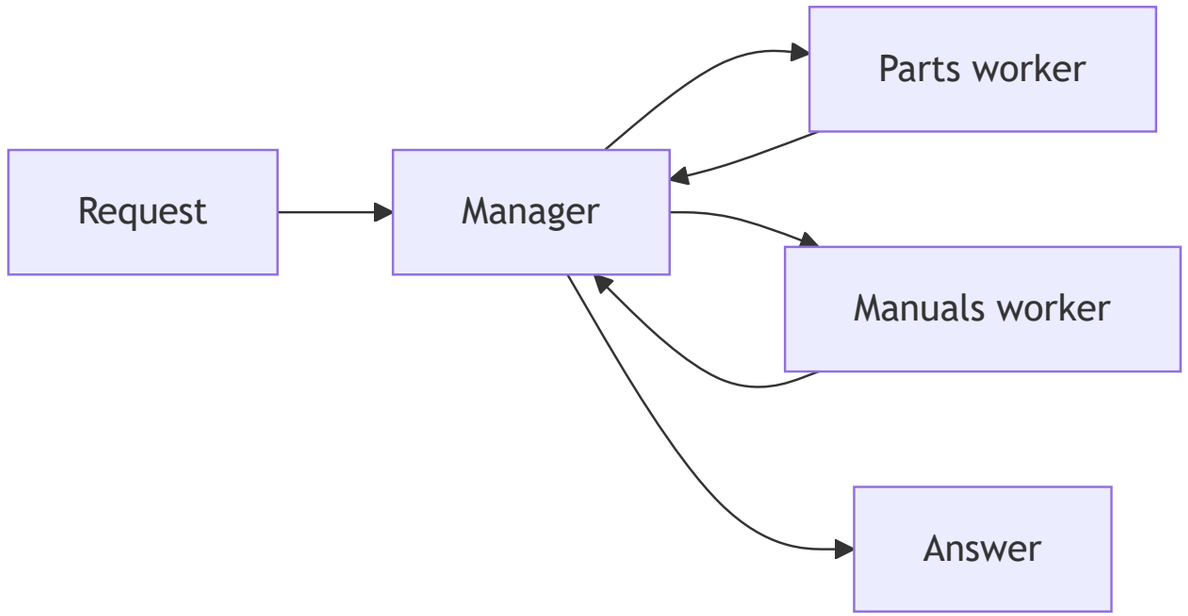
Subagent Context

- The worker starts with the delegated question and its own prompt
- Each worker retains its own model and tool exchanges
- The coordinator retains the task and returned answer
- Every fact crosses the boundary in the task or in the answer :
  - A task without fleet number CE12-2021-0117 can change the parts lookup
  - An answer without the 80 percent airflow condition changes the final advice
- A fresh worker invocation does not inherit earlier worker history

demo01b
subagent-context

Manager-Worker

- A manager owns task decomposition and the final answer
- Workers own bounded tasks and return findings
- Manager-worker can use agents as tools or explicit graph nodes
- Code controls the available workers and the stopping budget
- The manager decides whether a returned finding needs more work
- A lookup with known inputs, such as stock for CE-CF-02, stays an ordinary function



SpiralTrain developer courses

A Supervisor Node

- A supervisor node selects an unfinished worker and a task
- The selected worker receives that task in a separate context
- Each completed worker returns to the supervisor
- Findings are keyed by worker ; code caps the loop at two worker rounds

```
graph.add_edge(START, "supervisor")
for name in workers:
    graph.add_node(name, worker(name))
    graph.add_edge(name, "supervisor")
graph.add_conditional_edges("supervisor", lambda s: s["next"],
                            ["parts", "manuals", "finish"])
graph.add_edge("finish", END)
```

- A supervisor that finishes early or dispatches parts twice can leave out the manual's condition

demo02
supervisors

SpiralTrain developer courses

Hierarchical Teams

- Hierarchical orchestration nests one team inside another
- The outer manager receives the maintenance team's report
- The outer synthesis adds a context boundary and a model call ; 8 calls against 7 for the team alone
- A fact can drop out at either report

```
def manage(state):
    report = team.invoke(team_input(state["question"]), LIMIT)
    return {"report": report["answer"]}

manager.add_node("maintenance_team", manage)
manager.add_edge("maintenance_team", "manager")
```

demo02b
hierarchical-teams

SpiralTrain developer courses

Router

- A classifier selects specialists and writes their subquestions
- Dispatch executes the selected tasks
- A final node combines the specialist answers
- Classification occurs once per request in this graph
- A stock-only question selects only the parts worker
- A question about stock and fault F-CE-110 selects both workers
- Follow-up memory requires a surrounding conversation or checkpointer

demo03
router

Router Code

- `Send` supplies each selected branch with its own input

```
def dispatch(state):
    return [Send(r["agent"], {"question": r["query"]})
            for r in state["routes"]]

class RouterState(TypedDict):
    question: str
    routes: list[dict]
    answers: Annotated[list[str], operator.add]
    final: str

graph.add_conditional_edges("classify", dispatch,
                            ["parts", "manuals"])
```

- Each branch returns a new answer entry through the reducer
- Synthesis runs after the selected branches finish

demo03b
router-parallel

Delegation and Handoff

Property	Delegation	Handoff
Caller	Waits for the task result	Transfers the active role
Worker output	Returns to the coordinator	Continues the conversation
Typical context	Bounded task	Conversation with retained facts
Control	Coordinator chooses the next action	Active agent or step chooses it

- A parts lookup suits delegation
- Triage that collects a bus number can hand off to a specialist
- Either choice still has to settle how much history the receiver gets

Handoffs

- A tool changes the active agent through `Command.PARENT`
- The documented subgraph example forwards the last assistant message
- The probe exposes two faults in that transfer :
  - A stock result from the same turn is dropped ; the provider rejects the unanswered tool call
  - An earlier lookup is lost ; the receiver gives guidance and reports stock unavailable
- Seeded histories reproduce both faults against the live model

demo04
handoffs

SpiralTrain developer courses

Fixed Handoffs

- An agent that can transfer control makes one tool call at a time
- The transfer forwards the worker's full history and a matching receipt

```
return Command(
    goto=target, graph=Command.PARENT,
    update={"active_agent": target, "messages": [
        *runtime.state["messages"],
        ToolMessage(content="Transferred to " + target,
                    tool_call_id=runtime.tool_call_id)
    ]})
```

- With the lookups retained, the receiver answers stock, location and condition
- An agent can still stop before it transfers ; the earlier probe recorded one half answer in three

demo04b
handoffs-fixed

SpiralTrain developer courses

Handoff Middleware

- One agent keeps the conversation and the active step in state
- `record_bus` validates the fleet number and updates the step
- Middleware chooses the prompt and tools for each model call

```python
@wrap_model_call
def apply_step(request, handler):
    if request.state.get("current_step", "triage") == "triage":
        return handler(request.override(
            system_prompt="Collect the fleet number.",
            tools=[record_bus]))
    return handler(request.override(
        system_prompt="Answer using the tools.",
        tools=[part_stock, fault_guidance]))
```

- A checkpointer retains the step across user turns

demo04c
handoffs-one-agent

Skills

- A skill contains instructions loaded into the current agent's history
- Names and descriptions identify the available specialities

```python
@tool
def load_skill(name: str) -> str:
    """Load task instructions. Names: parts or manuals."""
    return SKILLS.get(name, "Unknown skill.")

skilled = agent_by_hand(llm,
    [load_skill, part_stock, fault_guidance],
    "Load each matching skill before using its tools.")
```

- The same agent keeps control and retains the loaded instructions
- The prompt asks for the skill first ; nothing stops a tool call without it

demo05
skills

SpiralTrain developer courses

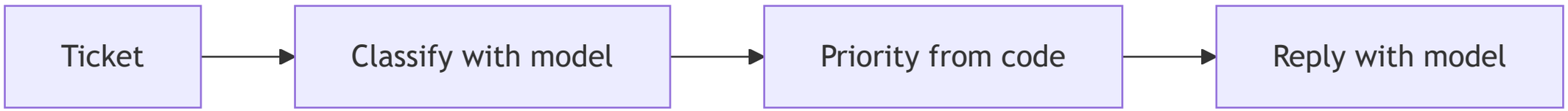
Specialized Agents

- **Specialisation can use a subagent or a skill**
- **A subagent isolates intermediate work :**
  - The worker keeps its own tool exchanges
  - The caller receives a bounded result
- **A skill retains intermediate work in the conversation :**
  - Follow-ups can reuse the loaded instructions and findings
  - Later calls process the accumulated context
- **The isolated stock worker reads 226 input tokens ; the skills agent 1,139 for the full question**
- **The single agent answers the full question with 435 input tokens**

demo05b
subagent-or-skill

Custom Workflow

- A LangGraph workflow combines agent steps with ordinary code
- The developer fixes dependencies and the order of execution
- Agent steps handle classification or writing
- Code applies known business rules and database lookups
- A workflow can contain subagents, routers or handoffs as nodes
- Each node exposes a state input and an update for inspection



SpiralTrain developer courses

Sequential Pattern

- A ticket follows classification, priority assignment and response
- Structured output restricts classification to the permitted labels
- Priority comes from a lookup table

```
def prioritise(state):
    return {"priority": {
        "complaint": "high", "question": "normal", "praise": "low"
    }[state["category"]]}

graph.add_edge(START, "categorise")
graph.add_edge("categorise", "prioritise")
graph.add_edge("prioritise", "respond")
graph.add_edge("respond", END)
```

- The prompt forbids a repair date ; the saved reply gives none

SpiralTrain developer courses

Parallel Agents

- Independent parts and manuals tasks can run concurrently
- Separate state keys retain their distinct results

```
graph.add_edge(START, "parts")
graph.add_edge(START, "manuals")
graph.add_edge(["parts", "manuals"], "compose")
graph.add_edge("compose", END)
```

- The join edge waits for both branches before composition
- Dynamic map-reduce uses `Send` and a reducer for collected results
- Tokens add up across the branches ; running them in parallel saves none
- The shared limiter cancels the speed-up : 42.4 seconds parallel, 41.8 sequential

demo06b
workflows-parallel

SpiralTrain developer courses

Orchestration in Code

- With CE-CF-02 and F-CE-110 known, no model has to choose a tool
- Code fetches the stock and error guidance
- A model composes the answer in one call, against five for the sequential agents

```
def fetch(state):
    return {
        "stock": part_stock.invoke({"part": "CE-CF-02"}),
        "guidance": fault_guidance.invoke({"error_code": "F-CE-110"})
    }

graph.add_edge(START, "fetch")
graph.add_edge("fetch", "compose")
graph.add_edge("compose", END)
```

- Code guarantees the scheduled lookups ; composition still needs checking

demo06c
orchestration-in-code

SpiralTrain developer courses

Collaborative Agents

- Collaborative agents work on one parts note or one repair decision
- Maker-checker separates writing the note from reviewing it
- Debate gives a judge two proposals, such as inspecting the fan or replacing it
- Generate and filter keeps only the notes that pass the database checks
- Every shape needs the same evidence, such as stock records and the manual
- Code sets the completion rule and the attempt cap
- Two models that agree can both be wrong ; the database settles a stock count

SpiralTrain developer courses

Maker-Checker

- A maker writes a parts note from database evidence
- A model checker returns approval and specific feedback
- A rejected draft returns to the maker with the feedback

```
graph.add_edge(START, "make")
graph.add_edge("make", "check")
graph.add_conditional_edges("check", lambda s:
    END if s["approved"] or s["rounds"] >= 2 else "make",
    ["make", END])
```

- Both roles need the required counts, locations and zero-stock warning
- Reaching the attempt cap can leave the draft unapproved
- The saved model checker approves a note the code checker rejects for a missing zero

demo07
maker-checker

Checkers Written in Code

- Queryable facts support deterministic checks :
  - Required part numbers, stock counts and locations
  - An explicit warning for zero stock
- A check returns specific repair findings
- The maker receives those findings on the next attempt
- The demo starts from a draft that marks CE-DA-03 available ; its stock is 0
- Formatting requirements can reject a factually correct paraphrase
- A pass proves only what `check_note` tests, such as identifiers, digits and locations
- Remaining findings stay visible when the attempt cap is reached

demo07b
checker-in-code

SpiralTrain developer courses

Debate with a Judge

- Two agents argue different first actions, inspecting the fan or replacing it
- Both agents receive the same manual excerpt and stock evidence
- A judge sees the proposals and the source evidence
- The judge returns its selected action and reason
- Six fans in stock do not justify a replacement
- The manual replaces the fan only below 80 percent airflow
- The saved judge selects inspection and keeps the airflow condition

demo08
debate-and-selection

SpiralTrain developer courses

Generate and Filter

- Independent calls write the parts note in different styles, bullets or a table
- Code checks each candidate against the database facts
- Invalid candidates retain their rejection findings
- Selection chooses the shortest passing candidate
- A stable role label breaks equal-length ties
- An empty set of passing candidates returns an explicit failure
- More candidates increase calls and tokens without guaranteeing a pass

```
return [Send("generate", {"task": style}) for style in
        ["Compact bullets.", "A compact table."]]
```

demo08b
generate-and-filter

Orchestration Shapes

- **Named shapes describe execution and selection policies :**

Shape	Bus maintenance application
Classify and act	Select the specialist for a ticket
Fan-out and synthesise	Combine stock and manual findings
Adversarial verification	Challenge a proposed replacement
Generate and filter	Keep notes that pass factual checks
Tournament	Compare surviving proposals in rounds
Loop until done	Repair until checks pass or the budget ends

- **Code defines coverage, joins and bounded termination**
- **Deep Agents packages delegation and context management in module 08**

Evaluating the Boundaries

- **Each boundary gets its own check :**
  - The specialist’s answer against the stock records
  - The delegated task for the fleet number and the condition asked for
  - The returned findings for facts lost on the way back
  - The final answer against the whole request
- **The comparison includes one agent with both tools**
- **A keyword check passes a sentence that contradicts itself**
- **Failures are read before a prompt changes or an agent is added**

SpiralTrain developer courses

Supervisor and Swarm Packages

- **Earlier prebuilt packages supplied multi-agent graphs :**
  - `langgraph-supervisor` centralised worker coordination
  - `langgraph-swarm` switched between active agents
- **Existing tutorials and applications still contain these packages**
- **The documented patterns expose the same control choices :**
  - A supervisor uses agents as tools or worker nodes
  - A swarm-style transfer uses handoffs and an active-agent key
- **Both shapes need only nodes, conditional edges and `Command`**

The Earlier Pattern Probe

- Luna medians on a stock-and-workshop question, 13 October 2026 :

Pattern	Correct	Calls	Input	Output	Seconds
Single agent	3/3	2	399	96	6.7
Subagents	3/3	6	781	204	20.3
Handoffs, fixes applied	2/3	4	1,165	99	13.5
Skills	3/3	4	1,036	131	14.6
Router	3/3	6	622	184	20.6
Custom workflow	3/3	5	472	129	17.0
Deep Agent, module 08	3/3	6	11,044	214	20.6

- Seconds include rate limiting at 0.3 requests per second
- The fixture is small and in memory, with stock for BR-220 and one workshop slot

SpiralTrain developer courses

What Each Pattern Costs

- Luna medians on the Veenhaven records, 14 October 2026 :

Pattern	Correct	Calls	Input	Output	Seconds
Single agent	3/3	2	435	113	17.9
Subagents	3/3	6	954	263	50.4
Handoffs, fixes applied	3/3	4	1,215	108	33.9
Skills	3/3	4	1,139	162	33.8
Router	3/3	6	816	247	50.0
Custom workflow	3/3	5	632	192	42.2
Code lookups	3/3	1	129	57	9.4

- Seconds include the shared limiter at 0.12 requests per second
- Code lookups use the known part and error-code arguments
- The single agent needs fewer calls and tokens than every multi-agent pattern

demo09
what-each-pattern-costs

SpiralTrain developer courses

Summary : Multi-Agent Patterns

- Multi-agent design controls what each agent sees and returns
- One agent is the baseline for a measured architecture decision
- Subagents isolate work ; skills retain it in one conversation
- Supervisor and manager-worker graphs coordinate bounded tasks
- Hierarchical teams introduce another context and output boundary
- Routers classify and dispatch ; handoffs change the active role
- Workflows fix sequential dependencies and parallel joins
- Collaborative patterns need evidence, checks and bounded completion
- Code handles known orchestration rules and deterministic checks
- Calls, tokens, seconds and answer quality determine the trade-off

- Introduction to LangGraph
- Graph Fundamentals
- State Management
- Building Agents
- Multi-Agent Patterns
- **Agent Communication**
- Advanced Workflows
- Deep Agents
- Production Deployment
- Real-World Applications

- Message Passing
- Shared State
- Full History
- Agent Context
- Handoffs
- Parent Message Update
- Shared Keys
- Transformed Input and Output
- Communication Protocols
- Agent Protocol
- MCP for Tools
- Served Agents
- Agent Cards
- A2A Tasks
- Two Agents over A2A
- Event Systems
- Subgraph Streaming
- v3 Projections
- Custom Events
- State Broadcasting
- Synchronization
- Conflict Resolution
- Conflicting Writes
- Voting
- Judge and Tie-Break
- Seller and Buyer
- Least Privilege

SpiralTrain developer courses

Message Passing

- **A message carries a selected payload :**
  - The sender names the recipient and a correlation id
  - The payload holds the facts that recipient needs
- **Shared state exposes every key the nodes have in common**
- **Veenhaven's stock and manual findings can travel either way**
- **A local envelope has no network transport**

Shared State

- Parallel nodes write different keys of one schema
- A join waits for both writers before the report

```
builder.add_edge(START, "parts")
builder.add_edge(START, "manuals")
builder.add_edge(["parts", "manuals"], "combine")
```

- `parts` writes stock ; `manuals` writes the error guidance
- The report node reads both keys

SpiralTrain developer courses

Full History

- Full history passes every message the caller holds
- The receiver can see the stock note and the open question
- Input grows with the transcript
- A later agent can repeat a lookup that is already in that history

SpiralTrain developer courses

Agent Context

- **Four boundaries for one maintenance request :**
  - Full history includes the earlier stock note
  - Task only sends the selected stock payload
  - A Python copy of the history adds a new task
  - A summary returns selected facts to the caller
- **The task-only request omits the first action on purpose**
- **A summary can drop a condition on the way back**

demo01b
full-history-task-or-summary

SpiralTrain developer courses

Handoffs

- A child node transfers control to a node in the parent

```python
def handoff(state):
    return Command(
        goto="manuals",
        graph=Command.PARENT,
        update={"messages": state["messages"]},
    )
```

- `Command.PARENT` targets the closest parent graph
- The destination is a node of the parent graph

demo02
subgraph-communication

SpiralTrain developer courses

Parent Message Update

- The parent merges messages by id with `add_messages`
- The receiver's list includes the stock note
- The written answer can still omit a fact from that list
- The module 05 tool handoff disables parallel tool calls
- A tool handoff needs a `ToolMessage` for the transfer call
- This node transfer has no pending tool call

SpiralTrain developer courses

Shared Keys

- A compiled subgraph shares keys present in both schemas
- `part_no` and `stock` cross into the parent
- `working_note` exists only on the child schema
- The parent result does not contain `working_note`

demo02b
shared-keys

SpiralTrain developer courses

Transformed Input and Output

- A wrapper node maps parent fields onto the child schema

```
def mapped_parts(state):
    child_input = {"part_no": state["item"]}
    child_output = parts_graph.invoke(child_input)
    return {"finding": child_output["stock"]}
```

- The child input has no `requester` field
- Leaving a field out of model input does not hide logs or credentials

demo02c
transformed-input-and-output

SpiralTrain developer courses

Communication Protocols

- Agent and tool interfaces use these protocols :

Boundary	Mechanism
One process	Tool call, shared keys, `Command`
Async subagent	Agent Protocol
Two servers	A2A JSON-RPC
Tools	MCP

- In-process agents can access the same process credentials
- Separate servers can use separate storage and credentials

SpiralTrain developer courses

Agent Protocol

- Agent Protocol is the HTTP API for runs and threads
- `POST /runs/wait` runs an assistant and returns its state
- Async subagents use these tools :
  - `start_async_task` and `check_async_task`
  - `update_async_task`, `cancel_async_task`, `list_async_tasks`
- Deep Agents owns that harness ; the server owns the transport

SpiralTrain developer courses

MCP for Tools

- `langchain.mcp` turns MCP tools into tools an agent can call
- The package is beta in langchain 1.4.5
- An MCP server publishes tools
- Agent-to-agent calls use A2A or Agent Protocol
- An agent card describes an assistant's communication interface

SpiralTrain developer courses

Served Agents

- `langgraph dev` serves a graph on the local Agent Server
- The echo server runs with the LangSmith variables removed
- `POST /runs/wait` accepts the graph id, such as `parts`
- The parts server sends metadata to the configured EU endpoint
- Both measured Studio banners use `smith.langchain.com`

demo03
served-agents

SpiralTrain developer courses

Agent Cards

- The card describes one assistant and its transport
- The installed server requires the assistant UUID :

```
GET /.well-known/agent-card.json?assistant_id=<uuid>
POST /a2a/<uuid>
```

- A graph name is rejected with ”must be a UUID”
- `POST /assistants/search` returns the UUID for that graph id
- Default assistant UUIDs derive from the graph id with `uuid5`
- A2A requires a `messages` key in the input schema
- The card grants no credentials

demo03b
agent-card

A2A Tasks

- The completed `SendMessage` response has `result.task` :
  - `status.state` is `TASK_STATE_COMPLETED`
  - Artifacts hold the assistant text
- `GetTask` returns the task directly, with history and no artifacts
- The completed `GetTask` status says ”Task completed successfully”
- `contextId` is the thread id and must be a UUID
- `message/send` unwraps the task and lowercases the state

SpiralTrain developer courses

Two Agents over A2A

- Two processes use ports 2124 and 2125 and separate storage
- The coordinator looks up the specialist UUID and posts JSON-RPC
- The notebook prints both cards and the stored exchange
- Each server allocates its own task id
- A follow-up sends a new message with the same `contextId`

demo04
two-agents-over-a2a

SpiralTrain developer courses

Event Systems

- A stream is an observation of a run
- `updates` carries the values a node returned
- A namespace names the subgraph that produced the update
- Custom events report progress to stream consumers
- A custom event does not update graph state

SpiralTrain developer courses

Subgraph Streaming

- `version="v2"` yields a dict with `type`, `ns` and `data`

```
for part in graph.stream(
    {"part_no": "CE-CF-02"},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
):
    print(part["ns"], part["data"])
```

- A non-empty `ns` marks the child node
- The parent update contains the keys the child returned

v3 Projections

- `stream_events(version="v3")` returns a `GraphRunStream`
- The caller drives the run by iterating a projection
- `run.subgraphs` yields a handle as each child starts
- Each child handle exposes a `values` projection
- v3 is experimental in langgraph 1.2.15

SpiralTrain developer courses

Custom Events

- `get_stream_writer` publishes a value beside the state update

```
writer = get_stream_writer()
writer({"event": "lookup-started", "part_no": state["part_no"]})
return {"finding": finding}
```

- The stream asks for `custom` and `updates`
- The custom payload is not written into state

demo05b
custom-events

SpiralTrain developer courses

State Broadcasting

- One node publishes a snapshot on a shared key
- Seller and buyer both read that key
- Each returns its own observation
- `operator.add` collects the observations
- The snapshot is the value published before the readers run

demo05c
broadcast-channel

SpiralTrain developer courses

Synchronization

- A join lists the nodes it waits for
- `defer=True` holds a node until other pending nodes finish
- The collector checks that parts and manuals both reported
- An incomplete set raises before `collected` is returned

```
builder.add_edge(["parts", "manuals"], "collect")
```

- The alternative uses separate edges and a deferred collector :

```
builder.add_node("collect", collect, defer=True)
builder.add_edge("parts", "collect")
builder.add_edge("manuals", "collect")
```

demo05d
fan-in-and-barriers

SpiralTrain developer courses

Conflict Resolution

- One owner can run agents with opposing goals
- Yair’s crew post puts a seller bot beside a buyer bot
- Yair calls the pair ”a conflict of interest i’ve chosen to ignore”
- Those bots share one computer and the same browser logins
- Code settles the seller’s and buyer’s proposals

SpiralTrain developer courses

Conflicting Writes

- Two nodes may write the same key in one step
- An unreduced key accepts one value per step
- A second value raises `InvalidUpdateError`
- A list with `operator.add` keeps both proposals
- The conflict stays visible for the settlement step

demo06
conflicts

SpiralTrain developer courses

Voting

- Independent roles vote on one proposal
- Each vote keeps the role, the action and the reason
- The reserve is a policy input supplied with the facts
- A majority is a count over the returned votes
- Shared evidence can make the votes agree for the same reason

demo06b
voting

SpiralTrain developer courses

Judge and Tie-Break

- A fixed tie supplies one sell vote and one retain vote
- The judge records an action and a reason
- Code selects the majority and retains stock on a tie

```
chosen = "sell" if sell_votes > retain_votes else "retain"
if chosen == "sell" and available - quantity < reserve:
    return "retain"
```

- A tie resolves to retain
- A majority that breaks the reserve is overridden

demo06c
judge-and-tie-break

SpiralTrain developer courses

Seller and Buyer

- Seller proposes a quantity to release
- Buyer proposes a quantity to keep
- Settlement keeps at least the reserve
- Retention is the greater of the reserve and the buyer's quantity
- The sale is capped by the seller's quantity and the remaining surplus
- The result records `external_action` as false

demo07
buyer-and-seller

SpiralTrain developer courses

Least Privilege

- Each agent receives the tool for its own draft
- `ToolRuntime` carries the principal into the tool
- The published tool schema contains `quantity` only
- `authorise` rejects the other role's operation
- Shared browser logins expose the same authenticated services

demo07b
least-privilege-per-agent

SpiralTrain developer courses

Summary : Agent Communication

- Message passing sends a selected payload ; shared state shares keys
- Handoffs cross graphs with `Command` and `Command.PARENT`
- Agent Protocol and A2A connect agents across processes
- MCP exposes tools an agent can call
- Event systems stream subgraphs, v3 projections and custom events
- State broadcasting publishes one snapshot for every reader
- Conflict resolution keeps both writes, then applies a vote, a judge or a tie rule
- Synchronization joins branches with fan-in and deferred nodes
- Least privilege gives each agent its own credentials and tools

- Introduction to LangGraph
- Graph Fundamentals
- State Management
- Building Agents
- Multi-Agent Patterns
- Agent Communication
- **Advanced Workflows**
- Deep Agents
- Production Deployment
- Real-World Applications

- Advanced Workflows
- Human-in-the-Loop
- The interrupt Function
- Resuming with Command
- Approval States
- Approve, Edit and Reject
- Several Interrupts in One Step
- Interrupts in Subgraphs
- Interrupts in Subagents
- Human-in-the-Loop Middleware
- Gating Agent Actions
- Loop Detection in Graphs
- A Watchdog Node
- Repeated Tool Calls
- Retry Strategies and Backoff
- Fallback Chains
- Subgraphs and Composition
- Conditional Branching at Scale
- Durable Execution
- Resuming After a Crash
- Graceful Workflow Drain

SpiralTrain developer courses

Advanced Workflows

- **A workflow can pause for a human decision :**
  - Before a part reservation or a log overwrite at the Veenhaven depot
  - Inside a subgraph or a subagent tool
- **A workflow can bound its own loops :**
  - A watchdog node stops a diagnosis that makes no progress
  - A guard blocks a repeated tool call
- **A workflow can recover from failure :**
  - Retries with backoff around a remote stock lookup
  - A fallback chain from live data to a snapshot to a manual queue
  - Resume from a checkpoint after a crash or a drain
- **Subgraphs compose workflows owned by different teams**

SpiralTrain developer courses

Human-in-the-Loop

- **An agent loop can act on a wrong conclusion :**
  - Order traction cooling fans that bus BW-7001 does not need
  - Overwrite a verified maintenance log without depot sign-off
- **A human-in-the-loop workflow pauses before such an action :**
  - A reviewer sees the proposed action or the draft
  - The paused state sits in the checkpointer, with no time limit
- **The graph continues only when the reviewer supplies a decision**

SpiralTrain developer courses

The interrupt Function

- `interrupt(payload)` pauses the graph inside a node :

```python
from langgraph.types import interrupt

def review(state: Reply):
    decision = interrupt({"draft": state["draft"], "options": ["approve", "edit", "reject"]})
    return {"action": decision["action"]}
```

- A pause requires a checkpointer to store the thread state and the payload
- The caller reads the payload from `out.interrupts[0].value`
- `graph.get_state(cfg).next` names the paused node

demo01
interrupts

SpiralTrain developer courses

Resuming with Command

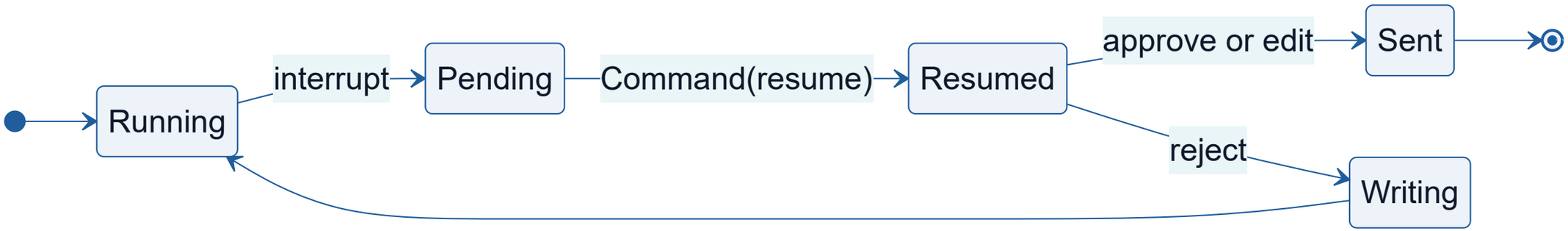
- `Command(resume=value)` continues the paused thread :

```
from langgraph.types import Command

graph.invoke(Command(resume={"action": "approve"}), cfg, version="v2")
```

- The paused node runs again from its first line :
  - `interrupt(...)` now returns the resume value
  - Statements before `interrupt(...)` execute a second time
- Side effects belong after the `interrupt(...)` call

Approval States



- **The reviewer's choice picks the next state :**
  - Approve sends the draft unchanged
  - Edit replaces the draft, then sends it
  - Reject returns feedback to the writing node

SpiralTrain developer courses

Approve, Edit and Reject

- The review node routes on the decision :

```python
def review(state: Reply) -> Command[Literal["send", "write"]]:
    decision = interrupt({"draft": state["draft"], "options": ["approve", "edit", "reject"]})
    if decision["action"] == "approve":
        return Command(goto="send")
    if decision["action"] == "edit":
        return Command(update={"draft": decision["text"]}, goto="send")
    if decision["action"] == "reject":
        return Command(update={"feedback": decision["text"]}, goto="write")
```

- `Command(update=..., goto=...)` updates state and routes in one return
- Edit replaces the draft without a model call
- Reject passes the reviewer's notes to `write` for a rewrite

demo01b
approve-edit-reject

SpiralTrain developer courses

Several Interrupts in One Step

- **Two `interrupt` calls in one node pause one after the other :**
  - LangGraph matches resume values to calls by their order
  - The order of the calls must not change between runs
- **Parallel nodes can pause in the same step :**
  - `out.interrupts` holds one `Interrupt` per node, each with its own `id`
  - The resume value maps each `id` to its reply :

```
replies = {item.id: f"approved {item.value['check']}" for item in out.interrupts}
checks.invoke(Command(resume=replies), check_cfg, version="v2")
```

demo01c
several-interrupts

SpiralTrain developer courses

Interrupts in Subgraphs

- A child graph compiles without a checkpointer and mounts as a node
- The parent's checkpointer stores the child's state
- An interrupt in the child reaches the parent caller :
  - `parent.invoke(...)` returns it in `out.interrupts`
  - `parent.get_state(cfg, subgraphs=True)` shows the paused child node
- Resuming the parent thread delivers the reply to the child

demo02
interrupts-in-multi-agent

SpiralTrain developer courses

Interrupts in Subagents

- **A subagent tool can invoke a child graph that pauses :**
  - The supervisor calls the inspection specialist tool
  - The child graph reaches `interrupt` and the pause surfaces at the supervisor
  - The supervisor graph is paused in its `tools` node
- **Resuming the supervisor runs the tool function again :**
  - Code before the child invocation executes a second time
  - The child receives the reply and the tool returns its `ToolMessage`

demo02b
in-a-subagent

SpiralTrain developer courses

Human-in-the-Loop Middleware

- `HumanInTheLoopMiddleware` names the tools that need approval :

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

policy = HumanInTheLoopMiddleware(interrupt_on={
    "reserve_part": {"allowed_decisions": ["approve", "edit", "reject"]}
})
agent = create_agent(llm, tools=[reserve_part], middleware=[policy],
                     checkpointer=InMemorySaver())
```

- A call to a listed tool pauses before the tool runs
- Tools missing from `interrupt_on` run without a pause

SpiralTrain developer courses

Gating Agent Actions

- The interrupt payload holds `action_requests` and `review_configs`
- Each action request carries the tool name and the model's arguments
- `Command(resume={"decisions": [...]})` returns one decision per request
- Each decision has a type :
  - `approve` runs the tool with the model's arguments
  - `edit` runs it with `edited_action` arguments, such as quantity 1
  - `reject` returns an error `ToolMessage` and the tool does not run

demo03
human-in-the-loop-middleware

SpiralTrain developer courses

Loop Detection in Graphs

- **A cyclic graph can run without making progress :**
  - A model repeats the same diagnostic search when the answer is missing
  - Two agents keep reversing each other's revisions
- **`recursion_limit` caps the number of supersteps in one run :**
  - Exceeding it raises `GraphRecursionError`
  - The run ends in an error, with no route to a person
- **A progress check in graph state stops a stalled run earlier**

SpiralTrain developer courses

A Watchdog Node

- A watchdog node counts steps without new evidence :

```
def watchdog(state: Investigation):
    stalled = state["stalled"] + 1 if state["evidence"] == state["previous"] else 0
    stop = stalled >= 2 or state["attempts"] >= 4
    return {"previous": state["evidence"], "stalled": stalled,
            "status": "manual review" if stop else "continue"}
```

- The watchdog runs after each diagnostic step
- Two stalled steps or four attempts route the run to manual review
- The run ends in the review queue before it reaches `recursion_limit`

demo04
loop-detection

SpiralTrain developer courses

Repeated Tool Calls

- A model can repeat the same tool call on consecutive turns
- A signature `(name, sorted_args)` identifies a repeated call
- A guard node checks each call before it reaches the database :
  - The first read runs and stores its signature in `state["seen"]`
  - A repeat returns an error `ToolMessage` and does not run
- The error message tells the model to answer from what it has

demo04b
repeated-tool-calls

SpiralTrain developer courses

Retry Strategies and Backoff

- **`RetryPolicy` from module 04 retries a node on a transient exception :**
  - Retry a timeout or an unavailable service
  - Fail a validation error at once ; `retry_on` selects the exceptions
- **Backoff spaces the attempts :**
  - `initial_interval` sets the first wait
  - `backoff_factor` multiplies the wait after each failure
  - `max_interval` caps the wait
  - `jitter` adds a random offset so workers do not retry in step

demo05
retries-and-fallbacks

SpiralTrain developer courses

Fallback Chains

- `with_fallbacks` tries each runnable in turn on a listed exception :

```
chain = RunnableLambda(live).with_fallbacks(
    [RunnableLambda(snapshot), RunnableLambda(manual)],
    exceptions_to_handle=(ConnectionError, LookupError)
)
```

- Each tier answers with less authority :
  - `live` reads current stock and may back a reservation
  - `snapshot` reads cached stock and marks the answer as possibly stale
  - `manual` opens a ticket for a person to check the shelf
- A fallback answer carries its source and does not authorize a reservation

demo05b
fallback-chain

SpiralTrain developer courses

Subgraphs and Composition

- **A subgraph lets one team build and test one stage :**
  - The diagnosis team owns the lookup workflow
  - The reporting team owns the report workflow
- **A compiled child graph mounts as a node in the parent :**
  - Keys in both schemas pass between parent and child
  - A wrapper function maps state when the child has its own schema
- **Each child graph can be tested and traced on its own**

demo06
composition-and-durability

SpiralTrain developer courses

Conditional Branching at Scale

- **A workflow with many specialist queues needs many branches :**
  - An `if/elif` chain in a router grows with every queue
  - A routing table keeps the rules apart from the nodes
- **`add_conditional_edges` maps each queue to a node via `path_map` :**

```
routes = {"F-CE": "electric", "F-MM": "diesel", "F-CD": "doors"}
path_map = {name: name for name in set(routes.values()) | {"manual"}}
branching.add_conditional_edges("classify", lambda s: s["queue"], path_map)
```

- **An unknown code routes to the `manual` queue**

demo06b
branching-at-scale

SpiralTrain developer courses

Durable Execution

- A checkpointer saves the state after every step
- `durability` sets when the save is written :
  - `"async"`, the default : while the next step runs
  - `"sync"` : before the next step starts
  - `"exit"` : only when the run ends
- A crash during a step leaves the earlier checkpoints intact
- A resumed thread continues from the latest checkpoint
- Completed steps do not run again

SpiralTrain developer courses

Resuming After a Crash

- A checkpointer preserves completed work before a node crashes :

```
with SqliteSaver.from_conn_string("workflow.sqlite") as saver:
    running = durable_builder().compile(checkpointer=saver)
    # Node 'prepare' succeeds, node 'finish' raises RuntimeError
    running.invoke(initial, crash_cfg, durability="sync")

with SqliteSaver.from_conn_string("workflow.sqlite") as saver:
    restarted = durable_builder().compile(checkpointer=saver)
    recovered = restarted.invoke(None, crash_cfg, durability="sync")
```

- `invoke(None, cfg)` runs only `finish` ; the model call in `prepare` is not repeated

demo06c
resuming-after-a-crash

SpiralTrain developer courses

Graceful Workflow Drain

- A killed worker can stop in the middle of a node
- `RunControl` from `langgraph.runtime` stops a run between nodes :
  - A signal handler or a node calls `request_drain(reason)`
  - The running node finishes and its checkpoint is saved
  - The run raises `GraphDrained` before the next node starts
- `invoke(None, cfg)` on a new worker runs the remaining nodes

SpiralTrain developer courses

Summary : Advanced Workflows

- `interrupt` pauses a node ; `Command(resume=...)` supplies the decision
- A paused node runs again from its first line on resume
- Interrupts in subgraphs and subagent tools reach the parent caller
- `HumanInTheLoopMiddleware` pauses calls to the tools it lists
- A watchdog node stops a stalled run ; a guard blocks a repeated call
- `RetryPolicy` retries with backoff ; `with_fallbacks` falls back by tier
- Subgraphs compose workflows ; a routing table scales branching
- A checkpointer resumes a run after a crash or a drain

exercise07
Approval in a Subagent

- Introduction to LangGraph
- Graph Fundamentals
- State Management
- Building Agents
- Multi-Agent Patterns
- Agent Communication
- Advanced Workflows
- **Deep Agents**
- Production Deployment
- Real-World Applications

- What is an agent harness?
- Framework, runtime, harness
- The agent loop
- A todo list
- Files in state
- A task tool
- create_deep_agent
- Planning with write_todos
- The virtual filesystem
- Offloading large results
- State backend
- Filesystem backend
- Store backend
- Composite backend
- Permissions
- The interpreter
- Interpreter limits
- A local shell
- Memory in AGENTS.md
- Skills
- Tools bound to skills
- Dict subagents
- CompiledSubAgent
- Fork mode
- The general-purpose subagent
- Dynamic subagents
- Recursive language models
- Orchestration shapes
- interrupt_on
- Summarisation
- Context limits
- Profiles
- Deep Agents and the patterns
- The Claude Agent SDK
- Codex

SpiralTrain developer courses

What is an agent harness?

- **An agent harness is the code around the model loop :**
  - It stores files, offers tools and decides when a run stops
  - The model still chooses the next tool call
- **`create_agent` is a small harness on the LangGraph runtime**
- **Deep Agents is a larger harness on `create_agent` :**
  - Files, delegation, skills, memory and context limits ship together
  - The caller still owns the task, the tools and the permissions

SpiralTrain developer courses

Framework, runtime, harness

- **Three layers sit under one Deep Agent :**
  - LangChain supplies the model, the tools and agent middleware
  - LangGraph runs the graph, the checkpointer and the store
  - Deep Agents adds files, the `task` tool and the context policy
- **The pin is `deepagents` 0.7.24, on `langchain` 1.4.5 and `langgraph` 1.2.15**
- **The same patterns from module 05 are still visible in the middleware stack**

SpiralTrain developer courses

The agent loop

```
plain = create_agent(llm, tools=[part_stock])
result = plain.invoke(ask(STOCK_TASK), LIMIT)
```

- The loop is model, tool, model
- One stock lookup on `openai/gpt-6-luna` :
  - `part_stock` ran once
  - 2 model calls, 194 input tokens, 45 output tokens
- The answer named 6 fans at Rietpoort
- The callback recorded 0 malformed tool calls

SpiralTrain developer courses

A todo list

```
planned = create_agent(
    llm, tools=[part_stock], middleware=[TodoListMiddleware()])
```

- `TodoListMiddleware` adds `write_todos`
- **The list records intended work :**
  - The run wrote one item, called `part_stock`, then marked it completed
  - The middleware does not enforce that order
- **The same lookup cost more context :**
  - 4 calls and 5502 input tokens
  - The plain agent used 2 calls and 194 input tokens

SpiralTrain developer courses

Files in state

```python
class ScratchState(AgentState):
    files: Annotated[dict[str, str], merge_files]
```

- Several tools write `files`, so the reducer merges dicts
- `write_note` returns a short confirmation :
  - The body stays under `files`
  - The message list does not have to carry it
- The lookup saved `/stock.md` and answered 6 fans at Rietpoort
- 3 model calls, 4160 input tokens, 0 malformed tool calls

SpiralTrain developer courses

A task tool

```python
@tool
def task(description: str) -> str:
    """Delegate a parts lookup."""
    return answer(worker.invoke(ask(description), LIMIT))
```

- The worker is its own `create_agent` call, with its own messages
- The parent receives the final sentence :
  - The worker's tool was `part_stock`
  - The parent's tools were `task` and `write_note`
- 5 model calls, 4438 input tokens, 0 malformed tool calls
- Intermediate tool results stayed in the worker trace

SpiralTrain developer courses

create_deep_agent

```
packaged = create_deep_agent(model=llm, tools=[part_stock])
```

- **Default tools in 0.7.24 :**
  - `ls`, `read_file`, `write_file`, `edit_file`, `delete`, `glob`, `grep`
  - `execute` and `task`
- **`write_todos` is absent until `TodoListMiddleware` is passed**
- **No caller prompt and no harness profile :**
  - The authored system prompt is empty
  - `BASE_AGENT_PROMPT` has been deprecated since 0.7.0
  - Tool schemas and middleware fragments still reach the model
- **The stock lookup called `part_stock` and `write_file` :**
  - 3 model calls, 6272 input tokens, 91 output tokens
  - `todos` stayed empty, and the callback recorded 0 malformed tool calls

demo02
create-deep-agent

SpiralTrain developer courses

Planning with write_todos

```
planner = create_deep_agent(
    llm, tools=[part_stock], middleware=[TodoListMiddleware()])
```

- Planning is opt-in on this pin
- Three Codex harness profiles are the built-in exception :
  - `openai:gpt-5.1-codex`, `openai:gpt-5.2-codex`, `openai:gpt-5.3-codex`
  - Each profile adds `TodoListMiddleware` because its prompt talks about todos
- `openai/gpt-6-luna` matches no harness profile, so the demo passes the middleware itself
- A completed item is still a claim ; the tool result is the check
- That lookup used 4 calls and 13290 input tokens, with 0 malformed tool calls

demo02b
write-todos

SpiralTrain developer courses

The virtual filesystem

```
inputs = {**ask("Read /manual.md and report the first action for F-CE-110."),
          "files": {"/manual.md": create_file_data(page)}}
```

- A path such as `/manual.md` names a backend file
- The notebook directory has no `manual.md` after the read
- `read_file` accepts `offset` and `limit` for a window of lines
- The seeded page is the CE-12 error-code manual

SpiralTrain developer courses

Offloading large results

- The default tool-result limit is 20000 tokens
- The demo lowers `tool_token_limit_before_evict` to 600 :
  - 31 manuals are 18795 characters on disk
  - The tool result was 19626 characters and was saved under `/large_tool_results/`
- The model saw the path and a preview, then reported 2 sources
- The printed length is the check ; the preview is too short to count the files

demo03b
offloading-large-results

SpiralTrain developer courses

State backend

```
state_agent = create_deep_agent(
    model=llm, backend=StateBackend(), checkpointer=InMemorySaver())
```

- `StateBackend` is the default
- Files for a thread live in that thread's graph state
- The checkpointer returns them on the next turn of the same thread
- A second `thread_id` starts without those files
- The memory checkpointer ends when the process ends

SpiralTrain developer courses

Filesystem backend

```
disk = FilesystemBackend(root_dir=disk_root, virtual_mode=True)
```

- `virtual_mode=True` maps `/note.md` to a file under `root_dir`
- A path with `..` or `~`, or a path outside the root, raises `ValueError`
- The message from this pin is `Path traversal not allowed`
- Those rules cover the file tools
- A shell command is outside this check, which demo05 takes up

demo04b
filesystem

SpiralTrain developer courses

Store backend

```
shared = StoreBackend(
    store=store, namespace=lambda _runtime: ("veen-service", "user-a"))
```

- This pin requires the `namespace` callback
- Files under one namespace are visible to later invokes that use it
- A second namespace reads the same path as missing
- `InMemoryStore` keeps the rows until the process exits
- A durable store would keep them across processes ; this demo does not start one

demo04c
store

SpiralTrain developer courses

Composite backend

```
mixed = CompositeBackend(
    default=StateBackend(), routes={"/memory/": shared})
```

- The longest matching prefix selects the backend
- `/memory/policy.md` is stored on the route as `/policy.md`
- A path with no matching prefix stays on the default, here state
- The file tools apply `permissions` ; a direct backend call skips them

demo04d composite

SpiralTrain developer courses

Permissions

```
rules = [
    FilesystemPermission(operations=["write"], paths=["/reports/**"], mode="allow"),
    FilesystemPermission(operations=["write"], paths=["/**"], mode="deny"),
]
```

- The first matching rule wins
- `mode` is `allow`, `deny` or `interrupt`
- An operation is `read` or `write`
- No matching rule means the call proceeds
- A subagent inherits the list, or replaces it with its own `permissions`

demo05
permissions-and-execution

SpiralTrain developer courses

The interpreter

```python
from langchain_quickjs import CodeInterpreterMiddleware
interpreter = CodeInterpreterMiddleware(
    ptc=[ticket_rows], subagents=False, mode="turn",
    memory_limit=16 * 1024 * 1024, timeout=2,
    max_ptc_calls=2, max_result_chars=1200)
analyst = create_deep_agent(model=llm, middleware=[interpreter])
```

- The model writes JavaScript in the `eval` tool
- `tools.ticketRows({})` loads ticket JSON through an allowed Python tool
- Filtering and summing run inside QuickJS ; only the result returns to the model
- Depot D01 tickets without parts total 4 tickets and 7.5 hours, matching Python
- QuickJS has no built-in host filesystem, network or process API
- An exposed tool still runs with the host's permissions

demo05b
the-interpreter

SpiralTrain developer courses

Interpreter limits

- **Limits apply to different parts of the run :**
  - `memory_limit` bounds the QuickJS heap
  - `timeout` counts VM execution time ; host calls can take longer
  - `max_ptc_calls` caps `tools.*` calls in one `eval`
  - `max_result_chars` truncates result and console output separately
- **REPL state persists by `mode` :**
  - `thread` across turns, `turn` within one turn, `call` for one invocation
- **PTC calls bypass per-tool parent approval :**
  - Gate `eval` or put approval in the called agent

SpiralTrain developer courses

A local shell

```
local = LocalShellBackend(
    root_dir=SHELL, virtual_mode=True, inherit_env=False)
```

- `execute` runs the host shell with `root_dir` as the working directory
- `virtual_mode` still confines the file tools to that directory
- `execute` can read and write outside it
- The command `echo veenhaven` returned `veenhaven`, exit code 0
- `inherit_env=False` leaves the process environment out of the command
- `SHELL` is `notebooks/08-deep-agents/workspace/shell/`
- `workspace/` is gitignored, so the demo's files stay out of the course tree

demo05c
a-local-shell

Memory in AGENTS.md

```
agent = create_deep_agent(model=llm, memory=["/AGENTS.md"])
```

- Each path is loaded at the start of the run
- The text is appended to the system prompt
- The file is Markdown ; HTML comments are removed before injection
- Several sources are concatenated in the order given
- Skill bodies stay out of that prompt until a skill is read or pinned

demo06
memory-and-skills

Skills

```
name: safety-briefing
description: Use when asked for safety rules before a depot visit.
```

- `skills=["/skills/"]` scans child folders for `SKILL.md`
- `name` must match the folder : lower case, digits and single hyphens, at most 64 characters
- The prompt receives the name and the description
- The instructions below the frontmatter stay in the file until `read_file`
- Two sources with the same name : the later source wins

demo06b
skills

SpiralTrain developer courses

Tools bound to skills

- **The 7 October 2026 skills change is in this pin :**
  - `metadata.include_tools` is a space-separated list of tool names
  - Those tools are passed to `SkillsMiddleware`
  - A listed tool stays off the model request until the skill is read, or named in `pinned_skills`
- **`pinned_skills` on `invoke` appends each skill body, then clears the list**
- **Set `skills_metadata` to `None` and the next run reads the files again**

```
metadata:
  include_tools: order_fan
```

demo06c
tools-bound-to-skills

SpiralTrain developer courses

Dict subagents

```
parts = {"name": "parts", "description": "Stock and location for one part.",
         "system_prompt": "Use part_stock.", "tools": [part_stock]}
team = create_deep_agent(model=llm, subagents=[parts])
```

- `task` takes `description` and `subagent_type`
- The default `mode` is `isolated` : the worker's messages are the description
- A `general-purpose` subagent is added when the caller did not pass that name :
  - It is offered the parent's tools
  - A harness profile can set `GeneralPurposeSubagentProfile(enabled=False)`

demo07
subagents

CompiledSubAgent

```
compiled = {"name": "manuals-graph",
            "description": "A compiled agent for one error code.",
            "runnable": manuals_graph}
```

- `runnable` is any graph whose state includes `messages`
- The harness does not wrap it in filesystem, skills or summarisation middleware
- The parent receives the last assistant text
- A non-empty `structured_response` is sent instead, as JSON
- `interrupt_on` from the parent is not copied onto this runnable

demo07b
compiled-subagent

SpiralTrain developer courses

Fork mode

- `mode="fork"` is experimental in 0.7.24
- The worker receives the parent messages, then the task description
- Its `system_prompt` is appended to the prompt it inherited
- A fork cannot declare `skills`
- A fork that calls `task` is refused ; the refusal text tells it to finish the work itself
- The fork run used 8752 input tokens ; the compiled run used 4375
- The task description left out F-CE-110
- The answer was still to check the traction cooling fan

```
forked = {"name": "manuals-fork", "description": "Looks up the error already named.",
          "tools": [fault_guidance], "mode": "fork"}
```

demo07c
fork-mode

SpiralTrain developer courses

The general-purpose subagent

- The built-in name is `general-purpose`
- The pin's description offers research, file search and multi-step tasks
- It is inserted when no subagent uses that name
- `enabled=False` on `GeneralPurposeSubagentProfile` stops the insertion
- With no synchronous subagent left, `create_deep_agent` does not register `task`
- Async subagents use a different middleware and stay available either way

SpiralTrain developer courses

Dynamic subagents

- Dynamic subagents dispatch configured workers from JavaScript, in beta
- The agent writes the fan-out and combines the returned objects :

```
const codes = ["F-CE-110", "F-CE-305"];
const schema = {type: "object", properties: {
  error_code: {type: "string"}, action: {type: "string"}
}, required: ["error_code", "action"]};
const found = await Promise.all(codes.map(code => task({
  description: "First action for " + code,
  subagentType: "manuals",
  responseSchema: schema
})));
JSON.stringify(found)
```

- `responseSchema` returns structured values for declarative subagents
- `max_subagent_concurrency=2` caps simultaneous dispatches per REPL
- The Python comparison puts the dispatch loop in the application

demo08
dynamic-subagents

SpiralTrain developer courses

Recursive language models

- **Zhang, Kraska and Khattab, MIT CSAIL, arXiv 2512.24601 :**
  - A long input stays in an environment the model inspects from code
  - The model is called on the pieces, and the pieces can be split again
- **The paper reports inputs on the order of 100 times the context window, on its tasks**
- **Zhang later describes two harness families :**
  - One keeps the trajectory in the prompt
  - One orchestrates with code and sub-calls
- **These 31 manuals fit in one call :**
  - One prompt used 5121 input tokens and 20 output tokens
  - Three calls used 5192 input tokens, and the largest used 2010
- **QuickJS loads the chunks through PTC and calls the same model as a reader :**
  - The root writes the dispatch script and receives compact findings
  - The complete chunks stay out of the root's model requests
  - Five calls used 15983 input tokens ; the largest reader used 2199

demo08b
splitting-a-long-input

Orchestration shapes

- **Many units of work, and a count known at run time, fit a loop in code :**
  - Classify and act : one label selects one specialist
  - Fan-out and synthesise : one call per unit, then one combined answer
  - Adversarial verification : a second pass challenges the first
  - Generate and filter : keep the candidates that pass a check
  - Tournament : compare candidates and keep a winner
  - Loop until done : repeat until a condition, with an explicit cap
- **A handful of fixed roles stays a dict of subagents**
- **The cap is the stopping rule for loop until done**

SpiralTrain developer courses

interrupt_on

```
gated = create_deep_agent(
    model=llm, interrupt_on={"write_file": True}, checkpointer=InMemorySaver())
gated.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config, version="v2")
```

- The pause is `HumanInTheLoopMiddleware` inside the harness
- A decision is `approve`, `edit`, `reject` or `respond`
- The graph needs a checkpointer, and resume continues the same thread
- A declarative subagent inherits the map unless it sets `interrupt_on` itself
- A `CompiledSubAgent` does not inherit it
- `FilesystemPermission` with `mode="interrupt"` builds the same pause for matching paths

demo09
control

SpiralTrain developer courses

Summarisation

- The default stack includes summarisation middleware
- Evicted messages are appended to `/conversation_history/{session}.md`
- `state["messages"]` keeps the original log ; the model sees the summary
- This model publishes no profile, so the default trigger is 170000 tokens and the keep window is 6 messages
- The demo replaces that middleware with a trigger of 3 messages so one lookup crosses it
- `compact_conversation` comes from `SummarizationToolMiddleware`, which the default stack does not add

demo09b
summarisation

SpiralTrain developer courses

Context limits

- **Three mechanisms run before a full summary :**
  - A tool result over 20000 tokens is evicted to the filesystem
  - A human message over 50000 tokens is evicted the same way
  - Old `write_file` and `edit_file` arguments can be clipped at a lower threshold
- **The summary call uses the agent model**
- **Fraction triggers apply when `max_input_tokens` is on the model profile**
- **`openai/gpt-6-luna`, constructed as `ChatOpenAI` for this course, has `profile is None`**

Profiles

- **Profiles are a beta API in two registries :**
  - A provider profile changes how `resolve_model` builds the chat model
  - A harness profile changes prompts, tool visibility and extra middleware after that
- **Built-in harness keys include the three Codex specs and Claude Haiku 4.5, Sonnet 4.6 and Opus 4.7**
- **Lookup for our instance uses provider `openai` and identifier `openai/gpt-6-luna`**
- **No key matches, so the profile is empty :**
  - There is no base prompt and no system-prompt suffix
  - There are no excluded tools and no extra middleware
- **Registering a profile is how a deployment would opt this model into planning or a shorter `task` list**

SpiralTrain developer courses

Deep Agents and the patterns

Harness feature	Module 05 pattern
`write_todos`	One agent ; the plan is state
Skills, `include_tools`	Skills, with tools scoped to the file
`task`, isolated	Subagents
`mode="fork"`	The worker sees the parent history
`CompiledSubAgent`	A custom workflow behind one tool
Fan-out in Python	A custom workflow ; the loop is code
`interrupt_on`	An approval gate on a tool

- **One desk question, both builders :**
  - Graph : 33 lines, 7 calls, 919 input tokens
  - Deep Agent : 8 lines, 6 calls, 11165 input tokens
  - Both named 6 fans at Rietpoort and the cooling-fan check

demo10
deep-agent-or-graph

SpiralTrain developer courses

The Claude Agent SDK

- The Agent SDK runs Claude Code's agent loop in the application's process
- The library includes file tools, a shell, subagents, skills, memory, permissions, sessions, hooks and MCP
- Python and TypeScript are the supported libraries
- The Client SDK calls the Claude API, and the application writes the tool loop
- Managed Agents run a hosted harness, with a cloud sandbox or a self-hosted sandbox
- Deep Agents is the harness in this course's process, on LangGraph, with the course model

Codex

- The Codex SDK starts and resumes a local Codex thread
- TypeScript package `@openai/codex-sdk` requires Node.js 18 or later
- Python package `openai-codex` talks to the local app server over JSON-RPC
- `thread.run` continues the thread ; a thread id resumes an older one
- Sandbox presets on the Python SDK :
  - `read_only` reads files
  - `workspace_write` writes inside the workspace and configured roots
  - `full_access` runs without those filesystem restrictions
- Deep Agents keeps the loop in-process and takes the model the caller built

SpiralTrain developer courses

The Academy course

- Introduction to Deep Agents, from LangChain Academy, is a free follow-up
- Six modules cover building, the execution environment, context, delegation, local deployment and Managed Deep Agents
- The public repo at `github.com/langchain-ai/lca-deepagents` has no licence
- This course keeps its own wording, its own examples and the Veenhaven data
- Their code pin is `deepagents` 0.7.0 ; this room is on 0.7.24

SpiralTrain developer courses

Summary : Deep Agents

- A harness packages files, skills, memory, subagents and context limits on `create_agent`
- Version 0.7 leaves the base prompt and `write_todos` to the caller, unless a profile adds them
- File tools, permission rules and `LocalShellBackend` are the execution environment in this pin
- Skills load on demand ; `include_tools` hides a tool until the skill is in context
- Isolated subagents see the task ; fork mode also sees the parent messages
- A long input can be split in code ; `interrupt_on` and summarisation bound a long run

SpiralTrain developer courses

- Introduction to LangGraph
- Graph Fundamentals
- State Management
- Building Agents
- Multi-Agent Patterns
- Agent Communication
- Advanced Workflows
- Deep Agents
- **Production Deployment**
- Real-World Applications

- Production Agent Services
- LangGraph API
- Agent Server Resources
- langgraph.json
- langgraph dev and Studio
- Threads and Runs in Practice
- Streaming to Clients
- Stream Modes and Subgraphs
- Double-Texting and Concurrency
- Persistence Backends
- Checkpoint Storage and TTLs
- DeltaChannel
- Async Subagents
- Async Task Lifecycle
- Co-deployed Graphs
- Firstmate Orchestration
- A2A on Agent Server
- Deployment Options
- Managed Deep Agents
- Scaling Strategies
- Evaluating a Multi-Agent System
- Final Answer Evaluation
- Trajectory Match
- Trajectory Judge
- Specialist and System Evaluation
- A LangSmith Experiment
- Monitoring
- Cost Optimisation
- Sandbox Return Channel

Production Agent Services

- **Agent services need operational controls :**
  - An HTTP API with threads, runs and streaming
  - Durable checkpoint storage shared by workers
  - Evaluation and monitoring on real traffic
  - Cost, concurrency and security controls
- **Agent Server serves LangGraph graphs and Deep Agents**
- **Veenhaven Bus exposes stock lookup and fault diagnosis graphs**
- **The service desk preserves conversation state across requests**

LangGraph API

- Agent Server exposes the LangGraph API over HTTP
- Core resources :
  - Assistants : a configured graph and its defaults
  - Threads : durable conversation state
  - Runs : one execution of an assistant on a thread
- The same API backs `langgraph dev`, self-hosted Agent Server and LangSmith Deployment
- Clients use `langgraph-sdk` ; Studio is a browser client of the same API

Agent Server Resources

- An assistant binds a `graph_id` from `langgraph.json` to runtime defaults
- A thread holds checkpointed state across runs
- A run applies input to an assistant on a thread and returns status and results
- Background runs support join, cancel and stream
- Follow-ups reuse the same thread id so prior messages remain available

SpiralTrain developer courses

langgraph.json

- The server loads graphs from a project configuration file :

```
{
  "dependencies": ["."],
  "graphs": {
    "service": "./graphs.py:service",
    "researcher": "./graphs.py:researcher",
    "supervisor": "./graphs.py:supervisor"
  },
  "env": "../../.env"
}
```

- Each entry points at a compiled graph export
- The server supplies checkpointing ; exported graphs omit a private checkpointer
- Dependencies install into the server environment from the project folder

langgraph dev and Studio

- Local development starts with :

```
uv run langgraph dev --no-browser --port 2134
```

- Studio opens against the server base URL in the browser
- EU accounts pass `--studio-url https://eu.smith.langchain.com`
- `/ok` reports readiness before the first client call
- Async subagents need concurrent run slots : `--n-jobs-per-worker`

SpiralTrain developer courses

Threads and Runs in Practice

- `assistants.search` lists the registered graphs
- `threads.create` gives the stock conversation a thread id
- `runs.wait` returns the stock lookup result
- A follow-up on the same thread can refer to CE-CF-02
- `runs.list` and `threads.get_state` show saved history
- The SDK is the integration surface for application code

demo01c
threads-and-runs

SpiralTrain developer courses

Streaming to Clients

- Clients consume server-sent events through the SDK stream
- Common stream modes :
  - `messages` / `messages-tuple` : model chunks with node metadata
  - `updates` : completed node writes
  - `values` : full state after each step
- `stream_subgraphs=True` includes nested graph events
- Clients display chunks before the run finishes

SpiralTrain developer courses

Stream Modes and Subgraphs

- `langgraph_node` identifies the model node for client filtering
- `stream_subgraphs=True` includes specialist events
- A list of stream modes requests tokens and node updates together
- `threads.get_state` reads the saved state after the stream ends

SpiralTrain developer courses

Double-Texting and Concurrency

- `multitask_strategy` controls a second run on a busy thread
- `reject` returns HTTP 409 while the first run occupies the thread
- `enqueue` waits for the current run, then starts
- `interrupt` stops the current run and retains its saved state
- `rollback` stops the current run and removes its checkpoints
- Provider quotas and worker capacity both bound concurrency

demo02b
double-texting

SpiralTrain developer courses

Persistence Backends

- Production Agent Server persists thread checkpoints in a database
- Local `langgraph dev` uses on-disk development storage under `.langgraph_api`
- PostgreSQL is the default production checkpoint backend
- Retention policies follow product and compliance requirements
- A Store holds application memory across threads

SpiralTrain developer courses

Checkpoint Storage and TTLs

- Shared checkpoint storage supports multiple worker processes
- TTL policies delete expired threads or prune older checkpoints
- Backups and access controls apply to thread data as to any customer store
- Development storage survives a local restart and is not a production database

demo02c
persistence-across-restarts

DeltaChannel

- `DeltaChannel` is beta in LangGraph 1.2.15
- Checkpoints store writes and periodic snapshots for reducer channels
- Replay applies ancestor writes to reconstruct accumulated state
- Incremental storage limits repeated writes of accumulated values
- Reducers must be deterministic and invariant to write batching

SpiralTrain developer courses

Async Subagents

- An `AsyncSubAgent` targets any Agent Protocol server
- Co-deployed graphs omit `url` and use in-process ASGI transport
- The parent must run asynchronously on Agent Server for ASGI transport
- Tools on the parent :
  - `start_async_task`, `check_async_task`, `update_async_task`
  - `cancel_async_task`, `list_async_tasks`
- Remote servers pass `url` and optional `headers`

demo03
async-subagents

Async Task Lifecycle

- `start_async_task` launches a background run and returns a task id
- `check_async_task` reads current status and results
- `update_async_task` interrupts the current run and starts a new run on its thread
- `cancel_async_task` stops a held or running task
- `list_async_tasks` reports tracked tasks, optionally filtered by status
- Updates retain the task id and replace the run id

SpiralTrain developer courses

Co-deployed Graphs

- Supervisor and worker graphs ship in one `langgraph.json`
- The supervisor is a Deep Agent with an `AsyncSubAgent` entry
- The worker is an ordinary compiled graph with a `messages` key
- The parent addresses the worker by `graph_id`
- Veenhaven Bus delegates stock research to the `researcher` graph

SpiralTrain developer courses

Firstmate Orchestration

- `firstmate` coordinates coding agents through processes and files
- One first mate dispatches each task to a crewmate in its own git worktree
- The orchestrator dispatches, watches, steers and cancels tasks
- State lives on disk ; a watcher wakes the orchestrator on events
- Separate worktrees isolate each coding agent’s changes

SpiralTrain developer courses

A2A on Agent Server

- Every assistant exposes `/a2a/{assistant_id}` where `assistant_id` is a UUID
- The agent card lives at `/.well-known/agent-card.json?assistant_id=...`
- Binding : A2A v1.0 JSON-RPC, send, stream, get, cancel and list tasks
- `contextId` identifies the Agent Server thread
- State must include a `messages` key
- `assistants.search` resolves graph names to assistant UUIDs

demo04
a2a-on-agent-server

Deployment Options

- Self-hosted Agent Server : your process, your database, your network
- LangSmith Deployment : managed Agent Server from the same `langgraph.json`
- Local `langgraph dev` matches the API surface used in deployment
- Operations ownership, region and compliance determine the hosting choice

SpiralTrain developer courses

Managed Deep Agents

- Hosted runtime in LangSmith for the Deep Agents harness
- Public beta ; the service runs on US LangSmith Cloud
- An agent is a folder : instructions, skills, subagents, tools
- CLI : `mda dev` locally, `mda deploy` to LangSmith
- The beta SDK is not open source
- The October plan records Plus at about $39 per seat per month

SpiralTrain developer courses

Scaling Strategies

- Stateless API processes share thread state through PostgreSQL
- Long runs belong in workers ; clients poll or subscribe
- Local concurrency uses `--n-jobs-per-worker`
- Production concurrency uses `N_JOBS_PER_WORKER`
- Provider quotas bound model requests across all workers
- More workers add run capacity within database and provider limits

Evaluating a Multi-Agent System

- **Four levels, from outside in :**
  - Final response : correctness and completeness
  - Single step : tool selection and arguments
  - Trajectory : tool-call sequence and evidence
  - Multi-turn : completion of the conversation goal
- **Specialist scores isolate errors before synthesis**
- **A correct specialist result does not guarantee a correct synthesis**

SpiralTrain developer courses

Final Answer Evaluation

- Code scorers check required facts against tool evidence
- Text checks miss false claims and reject valid paraphrases
- Expected stock and depot values come from the database

SpiralTrain developer courses

Trajectory Match

- `agentevals` compares live tool calls with a reference trajectory
- Four match modes on 0.0.9 :
  - `strict` : same message roles and matching calls per message
  - `unordered` : same tool-call coverage across messages
  - `subset` : outputs introduce no calls beyond the reference
  - `superset` : outputs include at least the reference calls
- Strict matching ignores prose and call order within one message

demo05b
trajectory-match

SpiralTrain developer courses

Trajectory Judge

- `create_trajectory_llm_as_judge` scores a full conversation with a rubric
- Judge input includes the task, tool calls, results and answer
- Human labels calibrate a judge used as a release gate
- Judges inherit model biases ; read the reasoning on a sample

demo05c
trajectory-judge

SpiralTrain developer courses

Specialist and System Evaluation

- Specialist evaluation checks its tools and facts
- System evaluation checks every required fact in the combined answer
- Failures separate into tool coverage, specialist quality and synthesis loss

demo05d
each-agent

SpiralTrain developer courses

A LangSmith Experiment

- The trainer experiment writes a dataset to EU LangSmith
- Code evaluators score a stock fact and `part_stock` coverage
- Account or permission failures are reported ; the local agent still runs
- Online evaluators reuse the same rubrics on sampled production traces

demo06
a-langsmith-experiment

Monitoring

- Run traces record model inputs, tool calls, results and failures
- LangSmith Trajectories show one chronological view across parent and subagents
- Online evals score a sample of production runs automatically
- Dashboards track cost per run, latency, error rate and judge scores
- Quality-score alerts flag regressions in sampled runs

SpiralTrain developer courses

Cost Optimisation

- Separate model instances support different agent token budgets
- Compact handoffs reduce synthesis input tokens in the stock demo
- Provider prompt caching reuses stable prefixes :
  - Instructions and tools precede changing content
  - Eligibility and retention depend on routing and the provider
- Prompt caching still sends a request ; it is not an application response cache
- Usage reports separate input, cached input and output tokens

demo07 cost

demo07b prompt-caching

SpiralTrain developer courses

Sandbox Return Channel

- Tool errors return to the agent for recovery
- Error text can contain an injected instruction
- Fixed error codes remove untrusted recovery text
- Application code validates the stock lookup outside the agent
- The reader has no egress tool or secret

demo08
injection-through-an-error

SpiralTrain developer courses

Summary : Production Deployment

- Agent Server serves assistants, threads and runs from `langgraph.json`
- Async subagents delegate background runs to worker graphs
- A2A exposes assistants through a separate protocol endpoint
- Clients stream events and read saved thread state
- Agent Server supports self-hosted and managed deployments
- Evaluations score final answers, trajectories and specialists
- Token measurements guide handoffs and prompt caching
- Error messages cross a trust boundary

SpiralTrain developer courses

- Introduction to LangGraph
- Graph Fundamentals
- State Management
- Building Agents
- Multi-Agent Patterns
- Agent Communication
- Advanced Workflows
- Deep Agents
- Production Deployment
- **Real-World Applications**

- Research Automation
- Research Evidence
- Customer Support
- Handoffs in Support
- Code Review
- Pipeline Checkers
- Data Analysis
- Content Pipelines
- Coding Agent Fleets
- A Personal Crew
- Firstmate Patterns
- Harness or Graph
- Swarms at the Frontier
- Navier-Stokes Swarm
- Fermat Formalisation
- Swarm Cost and Waste
- The Capstone Desk

SpiralTrain developer courses

Research Automation

- **Research agents turn open questions into sourced reports :**
  - Plan sub-questions, retrieve evidence and combine findings
  - Keep notes and drafts in files as context grows
- **`open_deep_research` is the reference implementation**
- **Parallel research workers receive bounded questions and separate source sets, as in module 05**

demo01
research-agent

Research Evidence

- **Every report claim needs a traceable source :**
  - Record the file or URL beside each finding
  - Report conflicting evidence and missing coverage
- **Files hold research notes and drafts outside the conversation, as in module 08**
- **Provenance checks confirm retrieval ; human review verifies whether the source supports the claim**

SpiralTrain developer courses

Customer Support

- A service agent routes, looks up policy and resolves or escalates a request :
  - Specialists hold distinct tools and authority
  - Irreversible actions pause for approval
- Handoffs carry the request between specialists, as in module 05
- Refunds, orders and account changes wait at approval gates from module 07

demo02
customer-support

Handoffs in Support

- **A handoff changes the active specialist and carries task state :**
  - The next agent needs earlier tool results and the request
  - Parallel tool calls can leave an unanswered call after the transfer
- **Shared state preserves the request and useful tool results across transfers, as in modules 05 and 06**
- **Evaluation checks complete answers and valid tool histories**

SpiralTrain developer courses

Code Review

- **Independent reviewers inspect different risks in the same change :**
  - Reviewers evaluate correctness, database safety and concurrency in parallel
  - A judge resolves duplicate or conflicting findings
- **Parallel review and a judge use the exploration pattern from module 05**
- **Evaluation from module 09 checks each reviewer and the combined findings for missed defects**

demo03
code-review

Pipeline Checkers

- **`no-mistakes` places a checker pipeline before changes reach `origin` :**
  - Stages run review, test, documentation, lint, push, pull request and CI
  - Mechanical findings are fixed automatically ; unresolved issues escalate
- **Defined pass conditions make the pipeline a maker-checker workflow from module 05**
- **Human approval controls consequential changes, as in module 07**

Data Analysis

- **An analytics agent turns a question into a query and an explanation :**
  - Keep the SQL query, returned rows and explanation visible together
  - Restrict the database connection to read-only access
- **A Deep Agent can use SQL tools and a configured execution sandbox from module 08**
- **Code checks compare reported figures with query results, using the checker pattern from module 05**

demo04
data-and-content

Content Pipelines

- A content pipeline transforms source material into reviewed output :
  - Retrieve the approved technical procedure
  - Draft and check required details before routing to editorial review
- Retrieval, drafting and validation form a fixed workflow from module 05
- Publication waits for human approval through an interrupt from module 07

demo04b
content-pipeline

SpiralTrain developer courses

Coding Agent Fleets

- **A fleet assigns separate coding tasks to isolated worktrees :**
  - One orchestrator dispatches work and receives events
  - Each worker receives a brief, session and writable checkout
- **Isolated subagents return findings to the orchestrator, as in modules 05 and 06**
- **Merge decisions use the human approval flow from module 07**

demo05
firstmate

A Personal Crew

- **Yair deployed about 30 single-job bots behind one first mate :**
  - Specialists handle taxes, vehicle status, portfolio, inbox and app administration
  - One coordinator routes requests to the relevant bot
- **”They all share one computer with the same browser logins, and nothing gets sent without my ok”**
- **Single-job workers have small tool sets behind one orchestrator, as in module 05**
- **Outbound actions wait for approval, as in module 07**
- **Shared logins motivate per-agent credentials and permissions from module 08**

Firstmate Patterns

firstmate component	course pattern
first mate and crewmates	orchestrator with isolated subagents, module 05
task worktree and session	context isolation and backend, module 08
watcher and event files	event driven execution, module 06
durable briefs and state	checkpointer and `Store`, modules 03 and 06
merge escalation	interrupt and approval, module 07

- Files and processes provide the coordination and persistence covered in modules 05 to 08

Harness or Graph

- **A coding harness fits work tied to a repository and long-lived sessions :**
  - Git worktrees, shell tools, watchers and persistent files form the runtime
  - The model interacts with existing files and developer tools directly
- **A graph fits application-owned state and explicit transitions :**
  - Typed schemas, state reducers and checkpointers define the boundaries
  - Conditional edges, loops and interrupt gates govern execution flow
- **Both runtimes can coordinate isolated workers, as in modules 05 and 08**
- **Application requirements determine who owns state and execution**

Swarms at the Frontier

- **Large mathematical searches distribute variants across isolated groups :**
  - Groups exchange messages internally to explore proof directions
  - A synthesiser consolidates useful findings across groups
- **Separate proof and disproof groups use parallel exploration from module 05**
- **Cross-group synthesis shares selected findings, as in module 06**

Navier-Stokes Swarm

- **OpenAI reports a coordinated search on an internal model :**
  - Groups explored different problem variants and proof or disproof directions
  - The winning group had on the order of 10,000 concurrent agents
  - Codex consolidated findings ; the model was updated mid-run
- **Lean formalisation checked the proposed proof after the search completed**
- **Parallel exploration and Lean verification combine worker and checker roles from module 05**

Fermat Formalisation

- **Anthropic reports formalising an existing proof of Fermat's Last Theorem :**
  - Dozens of parallel agents proved 30,300 theorems over 11 days
  - The final development used 29,500 proved theorems across 13 million lines of Lean
  - Lean's type checker rejected invalid proof steps mechanically
- **A shared theorem dependency graph coordinates parallel work, as in modules 05 and 06**

Swarm Cost and Waste

- **OpenAI reports about 130 billion output tokens for Navier-Stokes alone :**
  - The reported run generated 2.7 million messages over roughly 88 hours
  - Codex consolidated findings across groups throughout the run
- **swyx estimated about $40 million at public prices ; this is an outside estimate**
- **Alex Zhang stated, ”95% of the swarm is entirely useless” :**
  - He immediately hedged that the harness setup may not have mattered much in that run
  - The quote is his assessment, not a measured share of wasted tokens
- **Single-agent baselines from modules 05 and 09 test whether added workers improve quality enough to justify their cost**

The Capstone Desk

- **The Aurelia service desk compares a graph with a Deep Agent on one service dataset :**
  - Stage 3 implements parallel diagnosis, a judge, A2A communication and approval in a subagent
  - Stage 4 implements a Deep Agent with skills for procedures and specialists as subagents
- **Modules 05 to 08 supply the graph patterns, skills and subagents used in both stages**
- **Modules 08 and 09 evaluate both stages on accuracy, token cost and lines of code**

SpiralTrain developer courses

Summary : Real-World Applications

- Research agents split questions, retrieve evidence and cite findings
- Support agents hand off task state and pause consequential actions
- Parallel reviewers need a judge ; deterministic checkers enforce requirements
- Analysis and content pipelines keep queries and sources reviewable
- Coding fleets use isolated workers, durable state and event-driven orchestration
- Lean and code checks make model output mechanically rejectable
- Evaluation compares quality and cost with a simpler baseline